

Question 1: The definition of a table in the relational model does not contain:

Variante de răspuns

- A. the name of the table
- B. the name of the attributes
- C. the way the data will actually be stored

Răspuns corect

C. the way the data will actually be stored

Explicație

Modelul relațional este un model **logic**, care descrie structura datelor, constrângerile și relațiile dintre ele, dar face abstracție de modul fizic de stocare.

- **Definiția unei table (schema)** în modelul relațional include în mod obligatoriu **numele tablei** (varianta A) și **numele și tipurile atributelor/coloanelor** (varianta B). Aceste elemente definesc structura logică a tablei.
- **Modul în care datele sunt stocate fizic** pe disc (de exemplu, în ce fișiere, cum sunt organizate blocurile de date, ce structuri de indecși se folosesc - B-Tree, Hash, etc.) este un detaliu de **implementare** specific fiecărui Sistem de Management al Bazelor de Date (DBMS) și nu face parte din definiția logică a tablei în modelul relațional.

Question 2: A table in 1NF does not contain:

Variante de răspuns

- A. transitive dependencies
- B. null values
- C. repeating groups

Răspuns corect

C. repeating groups

Explicație

Definiția **Primei Forme Normale (1NF)** specifică regulile de bază pentru o bază de date relațională. O tabelă este în 1NF dacă îndeplinește următoarele condiții:

1. Fiecare celulă a tablei (intersecția dintre o coloană și un rând) conține o singură valoare (atomică).

2. Fiecare înregistrare (rând) este unică.

Pe baza acestei definiții:

- **Repeating groups (Grupuri repetitive):** O tabelă în 1NF **nu are voie** să conțină grupuri repetitive. Existența lor ar însemna că o singură celulă stochează mai multe valori (de exemplu, o listă de numere de telefon), ceea ce încalcă principiul atomicității.
- **Transitive dependencies (Dependențe tranzitive):** Acestea sunt permise în 1NF (și în 2NF). Eliminarea dependențelor tranzitive este obiectivul **Formei Normale 3 (3NF)**.
- **Null values (Valori nule):** Prezența valorilor nule este permisă în 1NF. Deși pot indica probleme de design în anumite contexte, ele nu încalcă regulile primei forme normale.

Question 3: A table in FN2 does not contain:

Variante de răspuns

- A. transitive dependencies
- B. functional dependencies on a subset of the key
- C. functional dependencies on any field

Răspuns corect

B. functional dependencies on a subset of the key

Explicație

Forma Normală 2 (FN2 sau 2NF) are două condiții:

1. Tabela trebuie să fie deja în Prima Formă Normală (1NF).
2. Toate atributele non-cheie (care nu fac parte din cheia primară) trebuie să fie **complet dependente** de întreaga cheie primară.

Această a doua regulă înseamnă că o tabelă în 2NF **nu are voie să conțină dependențe parțiale**.

- **Dependența parțială (Partial dependency)** apare atunci când un atribut non-cheie depinde doar de **o parte (un subset) a cheii primare compuse**, și nu de întreaga cheie. Varianta **B**, "**functional dependencies on a subset of the key**", este exact definiția unei dependențe parțiale. Prin urmare, o tabelă în 2NF nu poate conține așa ceva.

- **Dependențele tranzitive (Transitive dependencies)**, unde un atribut non-cheie depinde de un alt atribut non-cheie, sunt eliminate abia în **Forma Normală 3 (3NF)**. Deci, ele pot exista într-o tabelă 2NF.
- **Dependențele funcționale pe orice câmp (Functional dependencies on any field)** este o afirmație prea generală. Tabelele se bazează pe dependențe funcționale; scopul normalizării este de a le organiza corect (dependențe complete, non-tranzitive), nu de a le elimina complet.

Question 4

A table in FN2 is also in FN:

Variante de răspuns

- A. 1
- B. 3
- C. 3.5

Răspuns corect

A. 1

Explicație

Formele normale sunt ierarhice și cumulative. Regulile pentru a aduce o tabelă într-o formă normală superioară se bazează pe îndeplinirea condițiilor formelor normale inferioare.

- Pentru ca o tabelă să fie în **Forma Normală 2 (FN2)**, prima condiție este ca aceasta să fie deja în **Forma Normală 1 (FN1)**.
- Prin urmare, orice tabelă care se califică pentru FN2 este, prin definiție, deja în FN1.

O tabelă în FN2 nu este în mod obligatoriu și în FN3, deoarece ar putea conține încă dependențe tranzitive, care sunt eliminate abia în FN3. FN3.5 (cunoscută ca BCNF) este și mai restrictivă.

Question 5

A table in FN3 does not contain:

Variante de răspuns

- A. functional dependencies on any field
- B. transitive dependencies
- C. multivalued dependencies

Răspuns corect

B. transitive dependencies

Explicație

Definiția **Formei Normale 3 (FN3 sau 3NF)** se bazează pe eliminarea unui tip specific de dependență:

1. Tabela trebuie să fie în Forma Normală 2 (FN2).
2. Tabela nu trebuie să conțină **dependențe tranzitive**.

O **dependență tranzitivă** apare atunci când un atribut non-cheie (care nu face parte din cheia primară) depinde de un alt atribut non-cheie, în loc să depindă direct de cheia primară. Scopul principal al aducerii unei tabele la 3NF este tocmai eliminarea acestor dependențe indirecte.

- **Dependențele multivaloare (Multivalued dependencies)** sunt permise în 3NF. Eliminarea lor este obiectivul **Formei Normale 4 (FN4)**.
- **Dependențele funcționale (Functional dependencies)** sunt prezente și necesare; 3NF doar le organizează într-un mod mai strict, asigurând că toate attributele depind direct de cheie, și doar de cheie.

Question 6

Which is the highest FN that the table created below conforms to:

SQL

```
CREATE TABLE IF NOT EXISTS courses (
    project_id Int(11) NOT NULL,
    project_name varchar(40) NOT NULL,
    leader_name varchar(40) NOT NULL,
    leader_address varchar(40) NOT NULL
)
```

Variante de răspuns

- A. 1
- B. 2
- C. 3

Răspuns corect

B. 2

Explicație

Pentru a determina forma normală, trebuie să analizăm dependențele funcționale dintre atribute. Presupunem că `project_id` este cheia primară, deoarece un ID de proiect ar trebui să identifice în mod unic un proiect.

1. Analiza Dependențelor:

- `project_id` → `project_name` (ID-ul proiectului determină numele proiectului).
- `project_id` → `leader_name` (ID-ul proiectului determină numele liderului de proiect).
- `leader_name` → `leader_address` (Numele liderului determină adresa acestuia. Adresa depinde de lider, nu de proiect).

2. Verificarea Formelor Normale:

- **Este în 1NF (FN1)?** Da. Toate atributele (`project_id`, `project_name`, `leader_name`, `leader_address`) conțin valori atomice și nu există grupuri repetitive.
- **Este în 2NF (FN2)?** Da. Pentru a verifica 2NF, căutăm dependențe parțiale (unde un atribut non-cheie depinde doar de o parte a cheii primare). Deoarece cheia noastră primară (`project_id`) este simplă (nu este compusă din mai multe coloane), nu pot exista dependențe parțiale. Prin urmare, tabela este automat în 2NF.
- **Este în 3NF (FN3)?** Nu. Condiția pentru 3NF este să nu existe dependențe tranzitive. În acest tabel, avem o dependență tranzitivă clară:
 - `leader_address` depinde de `leader_name`.
 - `leader_name` depinde de cheia primară `project_id`.
 - Astfel, atributul non-cheie `leader_address` depinde de un alt atribut non-cheie, `leader_name`, ceea ce constituie o dependență tranzitivă (`project_id` → `leader_name` → `leader_address`). Aceasta încalcă regulile 3NF.

Deoarece tabela este în 2NF, dar nu și în 3NF, cea mai înaltă formă normală la care se conformează este **2NF (FN2)**.

Question 7

What does the following sequence return:

```
aes_decrypt(aes_encrypt(entity,key1),key2)
```

Variante de răspuns

- A. the original content of entity
- B. the content of entity encrypted with key2
- C. something else

Răspuns corect

C. something else

Explicație

Funcțiile `aes_encrypt` și `aes_decrypt` se bazează pe algoritmul de criptare simetrică AES (Advanced Encryption Standard). O caracteristică fundamentală a criptării simetrice este că **aceeași cheie** trebuie folosită atât pentru criptare, cât și pentru decriptare, pentru a recupera datele originale.

Să analizăm secvența pas cu pas:

1. **`aes_encrypt(entity, key1)`**: Conținutul original (`entity`) este criptat folosind o primă cheie, `key1`. Rezultatul este un text cifrat (ciphertext).
2. **`aes_decrypt(..., key2)`**: Se încearcă decriptarea textului cifrat obținut la pasul 1, dar folosind o a doua cheie, **diferită**, `key2`.

Deoarece cheia folosită pentru decriptare (`key2`) nu este aceeași cu cheia folosită pentru criptare (`key1`), procesul de decriptare va eșua. Rezultatul nu va fi conținutul original, ci un șir de date fără sens, corupte (garbage data). În unele implementări, funcția ar putea returna NULL sau o eroare, care se încadrează tot în categoria "altceva".

Prin urmare, răspunsul corect este **C. something else (altceva)**.

Question 8

Which command returns all the employees in all the projects as well as the project names without redundant information (an employee works on a single project):

Variante de răspuns

- A. SELECT employees.employee_name,project.project_name FROM employees JOIN projects ON employees.project_id = project.project_id
- B. SELECT employees.employee_name,project.project_name FROM employees FULL JOIN projects ON employees.project_id = project.project_id
- C. SELECT * FROM employees JOIN projects ON employees.project_id = project.project_id

Răspuns corect

A. SELECT employees.employee_name,project.project_name FROM employees JOIN projects ON employees.project_id = project.project_id

Explicație

Să analizăm cerința și opțiunile:

- **Cerința:** Se dorește o listă care conține numele angajaților și numele proiectelor la care lucrează. O condiție importantă este "without redundant information" (fără informații redundante).
- **Opțiunea A (JOIN):** Această comandă folosește un INNER JOIN (clauza JOIN este implicit INNER JOIN). Acesta combină rândurile din tabela employees cu cele din tabela projects doar acolo unde există o corespondență a project_id. SELECT specifică returnarea doar a celor două coloane de interes (numele angajatului și numele proiectului). Aceasta este metoda standard și eficientă pentru a obține exact informația cerută, fără date în plus.
- **Opțiunea B (FULL JOIN):** Un FULL JOIN (sau FULL OUTER JOIN) ar returna *toate* rândurile din ambele tabele. Ar include angajații care nu sunt alocați niciunui proiect (cu NULL în dreptul numelui proiectului) și proiectele care nu au niciun angajat alocat (cu NULL în dreptul numelui angajatului). Deși acest lucru poate fi util în alte scenarii, nu este ceea ce se cere aici ("employees in all the projects" se interpretează ca fiind perechile angajat-proiect existente).
- **Opțiunea C (SELECT *):** Clauza SELECT * cere bazei de date să returneze **toate** coloanele din ambele tabele. Acest lucru încalcă direct cerința "without redundant information", deoarece ar returna coloane de legătură (project_id din ambele tabele) și orice alte coloane existente, nu doar cele două nume solicitate.

Prin urmare, **Opțiunea A** este singura care îndeplinește corect și eficient toate condițiile din cerință.

Question 9

Which of the following commands returns all the students with an average grade over 8

Variante de răspuns

- A. `SELECT students.student_name,AVG(grades.grade) FROM students JOIN grades ON students.student_id = grades.student_id`
- B. `SELECT students.student_name,AVG(grades.grade) FROM students JOIN grades ON students.student_id = grades.student_id GROUP BY students.student_name HAVING AVG(grades.grade)>8`
- C. `SELECT students.student_name,AVG(grades.grade) FROM students JOIN grades ON students.student_id = grades.student_id WHERE AVG(grades.grade)>8`

Răspuns corect

B. `SELECT students.student_name,AVG(grades.grade) FROM students JOIN grades ON students.student_id = grades.student_id GROUP BY students.student_name HAVING AVG(grades.grade)>8`

Explicație

Această întrebare testează diferența fundamentală dintre clauzele WHERE și HAVING în SQL.

1. **Scopul:** Trebuie să calculăm o valoare agregată (media notelor, `AVG(grades.grade)`) pentru fiecare student și apoi să filtrăm rezultatele pentru a afișa doar studenții a căror medie este mai mare ca 8.
2. **GROUP BY:** Pentru a calcula media pentru *fiecare* student, trebuie mai întâi să grupăm toate notele în funcție de student. Acest lucru se face cu `GROUP BY students.student_name`.
3. **WHERE vs. HAVING:**
 - Clauza `WHERE` este folosită pentru a filtra înregistrările (rândurile) **înainte** ca acestea să fie grupate și agregate. Nu se poate folosi o funcție de agregare (precum `AVG()`, `SUM()`, `COUNT()`) într-o clauză `WHERE`. Din acest motiv, **Opțiunea C este incorectă** sintactic și va genera o eroare.
 - Clauza `HAVING` este folosită pentru a filtra **după** ce înregistrările au fost grupate și funcțiile de agregare au fost calculate. Este special concepută pentru a pune condiții pe rezultatele funcțiilor de agregare.

Analiza opțiunilor:

- **Opțiunea A:** Este incompletă. Calculează o medie, dar fără GROUP BY ar calcula media globală a tuturor notelor (sau ar genera o eroare în majoritatea sistemelor SQL, deoarece se selectează un câmp non-agregat alături de unul agregat). Nu are nicio clauză de filtrare pentru medie.
- **Opțiunea B:** Este corectă. Folosește GROUP BY pentru a crea grupuri pentru fiecare student, calculează AVG(grades.grade) pentru fiecare grup și apoi folosește HAVING pentru a filtra și a păstra doar grupurile (studenții) a căror medie este mai mare ca 8.
- **Opțiunea C:** Este incorectă. Încearcă să folosească AVG() într-o clauză WHERE, ceea ce nu este permis în SQL.

Question 10

Which of the following commands can have their effects reversed by rolling back a transaction:

Variante de răspuns

- A. delete
- B. truncate
- C. drop

Răspuns corect

A. delete

Explicație

Această întrebare se referă la diferența dintre comenzile DML (Data Manipulation Language) și DDL (Data Definition Language) și modul în care acestea interacționează cu tranzacțiile în SQL.

- **Tranzacțiile și ROLLBACK:** O tranzacție este o unitate logică de lucru. Comanda ROLLBACK anulează toate modificările făcute de la începutul tranzacției.
- **DELETE (Opțiunea A):**
 - Este o comandă **DML (Data Manipulation Language)**.
 - Șterge rândurile dintr-o tabelă unul câte unul și înregistrează fiecare ștergere în jurnalul tranzacției (transaction log).
 - Deoarece operațiunea este înregistrată în jurnal ca parte a tranzacției, efectele sale pot fi complet anulate printr-o comandă ROLLBACK.

- **TRUNCATE (Opțiunea B):**

- Este o comandă **DDL (Data Definition Language)**.
- Golește o tabelă de toate rândurile sale prin dealocarea paginilor de date, ceea ce este mult mai rapid decât DELETE.
- Majoritatea sistemelor de baze de date (Oracle, SQL Server, etc.) execută TRUNCATE cu un **commit implicit** (auto-commit). Acest lucru înseamnă că operațiunea este finalizată imediat și nu poate fi anulată cu ROLLBACK.

- **DROP (Opțiunea C):**

- Este o comandă **DDL (Data Definition Language)**.
- Șterge complet un obiect din baza de date (cum ar fi o tabelă).
- La fel ca TRUNCATE, DROP are un **commit implicit** și, prin urmare, efectele sale sunt permanente și nu pot fi anulate cu ROLLBACK.

Prin urmare, doar comanda **DELETE** poate fi "anulată" printr-un ROLLBACK într-o tranzacție.

Question 11

The following command

```
CREATE USER 'andrei'@'localhost' IDENTIFIED BY 'ism';
```

allows connections from:

Variante de răspuns

- A. only a remote computer
- B. only localhost
- C. any computer

Răspuns corect

B. only localhost

Explicație

În sistemele de baze de date precum MySQL sau MariaDB, sintaxa pentru crearea unui utilizator este CREATE USER 'nume_utilizator'@'host'.

- **'nume_utilizator'**: Este numele de login al utilizatorului (în acest caz, 'andrei').

- **'host'**: Specifică de unde (de la ce adresă/mașină) are voie acest utilizator să se conecteze.

În comanda dată, host-ul este specificat ca **'localhost'**. Acesta este un nume special care se referă întotdeauna la mașina locală, adică la același computer pe care rulează serverul de baze de date.

Prin urmare, utilizatorul 'andrei'@'localhost' va putea stabili o conexiune la baza de date **doar** dacă scriptul sau aplicația care inițiază conexiunea rulează pe același server cu baza de date.

- Pentru a permite conexiuni de la un computer la distanță (remote), s-ar specifica adresa IP a acelui computer (ex: '192.168.1.100').
- Pentru a permite conexiuni de la orice computer, s-ar folosi wildcard-ul '%' (ex: CREATE USER 'andrei'@'%' ...).

Question 12

The current host has the ip 10.2.3.4 After the following commands

SQL

GRANT select,insert,update on examples.* TO 'andrei'@'localhost';

REVOKE insert,update on examples.* FROM 'andrei'@'10.2.3.4';

FLUSH PRIVILEGES;

the user can:

Variante de răspuns

- A. select data from the "examples" table
- B. insert data into the "examples" table
- C. drop the examples table

Răspuns corect

A. select data from the "examples" table

Explicație

Să analizăm efectul fiecărei comenzi în contextul dat:

1. **Contextul crucial:** Ni se spune că utilizatorul se conectează de pe host-ul cu IP-ul **10.2.3.4**. Acest lucru înseamnă că doar regulile de permisiuni definite pentru utilizatorul **'andrei'@'10.2.3.4'** sunt relevante.
2. **GRANT select,insert,update on examples.* TO 'andrei'@'localhost';**

- Această comandă acordă permisiuni. Însă, ea se aplică **doar** utilizatorului 'andrei' care se conectează de pe '**localhost**'.
- Deoarece conexiunea curentă este de la 10.2.3.4, această comandă este **irelevantă** pentru situația de față și acționează ca un element de distragere a atenției.

3. **REVOKE insert,update on examples.* FROM 'andrei'@'10.2.3.4';**

- Această comandă **revocă (elimină)** permisiunile INSERT și UPDATE de la utilizatorul 'andrei'@'10.2.3.4'.
- Acest lucru înseamnă că, după executarea comenzii, utilizatorul conectat de la 10.2.3.4 **nu mai poate** insera sau actualiza date. Prin urmare, **Opțiunea B este incorectă.**

4. **Ce permisiuni rămân?**

- Întrebarea este formulată astfel încât să sugereze că utilizatorul 'andrei'@'10.2.3.4' avea un set de permisiuni *înainte* de executarea acestor comenzi.
- Comanda REVOKE a eliminat INSERT și UPDATE, dar **nu a menționat SELECT.**
- Scenariul cel mai logic (specific acestui tip de întrebări-capcană) este că utilizatorul 'andrei'@'10.2.3.4' avea inițial permisiunile SELECT, INSERT și UPDATE. După revocarea celor două, i-a rămas doar permisiunea SELECT.
- Permisiunea DROP (Opțiunea C) este o permisiune distinctă și foarte puternică, care nu a fost acordată și nu este afectată de comenzile de mai sus.

Prin urmare, singura acțiune pe care utilizatorul o poate efectua din lista de opțiuni este **selectarea datelor.**

Question 13

What does the following command

SELECT name FROM projects;

return:

Variante de răspuns

- A. the names of the projects in ascending order
- B. the names of the projects in descending order

- C. the names of the projects in the order they were inserted in

Răspuns corect

C. the names of the projects in the order they were inserted in (cu o mențiune importantă)

Explicație

Aceasta este o întrebare care, din punct de vedere tehnic, este o capcană, dar are un răspuns intenționat în contextul testelor.

- **Standardul SQL:** Conform standardului SQL, dacă o interogare (SELECT) **nu** include o clauză **ORDER BY**, ordinea în care rândurile sunt returnate este **nedeterminată și negarantată**. Sistemul de management al bazelor de date (DBMS) este liber să returneze rândurile în orice ordine consideră eficientă (de exemplu, ordinea de pe disc, ordinea dată de un index, etc.).
- **Analiza opțiunilor:**
 - **Opțiunea A (ordine crescătoare):** Este incorectă. Pentru a garanta ordinea alfabetică (ascendentă), comanda ar trebui să fie `SELECT name FROM projects ORDER BY name ASC;`
 - **Opțiunea B (ordine descrescătoare):** Este incorectă. Pentru a garanta ordinea alfabetică inversă (descendentă), comanda ar trebui să fie `SELECT name FROM projects ORDER BY name DESC;`
 - **Opțiunea C (ordinea în care au fost inserate):** Aceasta este, de asemenea, **tehnic incorectă** și negarantată. Deși într-o tabelă simplă, nou creată, rezultatele pot *parea* să fie returnate în ordinea inserării, acest comportament se poate schimba oricând după operațiuni de UPDATE, DELETE sau mentenanță a bazei de date.
- Concluzia în contextul testului:

Deoarece opțiunile A și B sunt în mod cert greșite (necesită ORDER BY), iar opțiunea "ordine aleatorie/negarantată" nu este disponibilă, opțiunea C este răspunsul intenționat. În multe medii academice, "ordinea de inserare" este folosită ca o prescurtare pentru "ordinea naturală/nesortată a datelor", în contrast cu ordinea alfabetică explicită.

Question 14

Which of the following SQL functions is not an aggregate function:

Variante de răspuns

- A. COUNT
- B. AVG
- C. TRIM

Răspuns corect

C. TRIM

Explicație

Întrebarea face distincția între funcțiile de agregare și funcțiile scalare (sau single-row) în SQL.

- **Funcțiile de agregare (Aggregate functions)** operează pe un set de valori (de exemplu, o coloană întreagă sau un grup de rânduri definite de GROUP BY) și returnează o singură valoare sumară.
 - **COUNT() (Opțiunea A)** este o funcție de agregare. Ea numără rândurile dintr-un set și returnează un singur număr.
 - **AVG() (Opțiunea B)** este o funcție de agregare. Ea calculează media aritmetică a unui set de valori numerice și returnează un singur rezultat.
 - Alte exemple includ SUM(), MIN(), și MAX().
- **Funcțiile scalare (Scalar functions)** operează pe o singură valoare de intrare (dintr-un singur rând la un moment dat) și returnează o singură valoare de ieșire.
 - **TRIM() (Opțiunea C)** este o funcție scalară pentru prelucrarea șirurilor de caractere (string). Ea elimină spațiile (sau alte caractere specificate) de la începutul și/sau sfârșitul unui singur string. De exemplu, TRIM(' test ') returnează 'test'. Deoarece operează pe o singură valoare, nu pe un set, **nu** este o funcție de agregare.

Prin urmare, TRIM este răspunsul corect.

Question 15

What does the following command

```
SELECT category,AVG(budget) FROM projects HAVING budget > 1000
```

return:

Variante de răspuns

- A. All the project categories with an average budget over 1000

- B. All the project categories
- C. Although it runs, the command contains a logical error

Răspuns corect

C. Although it runs, the command contains a logical error

Explicație

Comanda SQL furnizată conține câteva erori fundamentale de sintaxă și logică, motiv pentru care opțiunea C este cea mai potrivită.

Să analizăm erorile:

1. **Lipsa clauzei GROUP BY:** Atunci când într-o comandă SELECT se folosește o funcție de agregare (aici, AVG(budget)) alături de o coloană non-agregată (aici, category), standardul SQL impune folosirea unei clauze GROUP BY. Comanda ar fi trebuit să includă GROUP BY category pentru a calcula corect media bugetului pentru fiecare categorie în parte. Fără aceasta, majoritatea sistemelor de baze de date vor genera o eroare.
2. **Folosirea incorectă a clauzei HAVING:**
 - Clauza HAVING este destinată filtrării rezultatelor **după** ce gruparea și agregarea au avut loc. Se folosește pentru a pune condiții pe valorile agregate (de ex., HAVING AVG(budget) > 1000).
 - Pentru a filtra rândurile individuale **înainte** de grupare, se folosește clauza WHERE (de ex., WHERE budget > 1000).
 - Comanda din întrebare folosește HAVING pentru a filtra pe o coloană non-agregată (budget), ceea ce este o eroare logică; WHERE ar fi fost clauza corectă pentru acest tip de filtrare.

Concluzie:

Comanda este greșită din punct de vedere conceptual. Opțiunea A descrie ce ar face o comandă scrisă corect (... GROUP BY category HAVING AVG(budget) > 1000), nu comanda dată. Deoarece comanda este fundamental greșită, răspunsul corect este C, care recunoaște existența unei erori logice.

(Notă: Chiar dacă unele versiuni mai vechi sau mai permissive de MySQL ar putea "rula" comanda fără a genera o eroare, rezultatul ar fi unul arbitrar și fără sens, confirmând astfel eroarea logică.)

Question 16

A ResultSet object contains:

Variante de răspuns

- A. A connection
- B. An SQL command
- C. The result of the execution of an SQL command

Răspuns corect

C. The result of the execution of an SQL command

Explicație

Această întrebare se referă la conceptele de bază din API-urile de programare pentru baze de date, cum ar fi JDBC (Java Database Connectivity) sau ADO.NET.

Fluxul standard de interacțiune cu o bază de date dintr-o aplicație este următorul:

1. Se stabilește o **conexiune (Connection)** la baza de date.
2. Se creează un obiect de tip **comandă (Statement sau PreparedStatement)**, care încapsulează comanda SQL ce urmează a fi executată.
3. Se execută comanda. Dacă este o interogare de tip SELECT, baza de date returnează un set de date.
4. Acest set de date este încărcat într-un obiect special, numit **ResultSet**.

Prin urmare:

- Un obiect ResultSet reprezintă setul de rezultate (datele sub formă de tabel) returnate în urma executării cu succes a unei comenzi SQL de interogare. Acesta conține rândurile și coloanele returnate și oferă metode pentru a itera prin ele și a extrage datele.
- **Opțiunea A** este incorectă. Conexiunea (Connection) este obiectul care gestionează legătura cu baza de date, nu rezultatele.
- **Opțiunea B** este incorectă. Comanda SQL (An SQL command) este conținută într-un obiect Statement, nu în ResultSet.

Răspunsul corect este **C**, deoarece definește exact rolul unui obiect ResultSet.

Question 17

When using keys generated by the database, they can be obtained through the ____ object

Variante de răspuns

- A. Statement

- B. Driver
- C. Connection

Răspuns corect

A. Statement

Explicație

Această întrebare se referă la modul în care o aplicație poate recupera cheile primare auto-generate (de exemplu, o coloană cu AUTO_INCREMENT în MySQL sau IDENTITY în SQL Server) după executarea unei comenzi INSERT.

În API-urile de baze de date precum JDBC (Java Database Connectivity), procesul este următorul:

1. Se folosește obiectul Connection pentru a crea un obiect PreparedStatement sau Statement. Când se creează acest obiect, i se poate specifica un flag special (Statement.RETURN_GENERATED_KEYS) pentru a-i semnala driver-ului că dorim să recuperăm cheile generate.
2. Se execută comanda INSERT folosind obiectul Statement.
3. **După execuție**, se apelează metoda **getGeneratedKeys()** direct pe obiectul **Statement** (sau PreparedStatement). Această metodă returnează un ResultSet care conține cheia (sau cheile) nou create de baza de date.

Prin urmare, obiectul prin intermediul căruia se obțin aceste chei este **Statement**.

- **Opțiunea B (Driver)** este incorectă. Driver-ul este componenta care implementează protocolul de comunicare cu baza de date, dar aplicația nu interacționează direct cu el pentru a obține rezultate.
- **Opțiunea C (Connection)** este incorectă. Connection reprezintă sesiunea cu baza de date și este folosit pentru a crea obiecte Statement, dar nu pentru a prelua rezultatele specifice ale unei singure comenzi.

Question 18

The JDBC driver for MySQL cannot be loaded through

Variante de răspuns

- A. the command line
- B. loading the driver class in the class loader
- C. registering a Driver instance with the DriverManager
- D. all of the above

Răspuns corect

C. registering a Driver instance with the DriverManager

Explicație

Aceasta este o întrebare tehnică subtilă care face distincția între "încărcarea" (loading) unei clase și "înregistrarea" (registering) unui driver.

Să analizăm metodele prin care DriverManager poate deveni conștient de un driver JDBC:

1. **Încărcarea automată (JDBC 4.0+)**: Driver-ele moderne, inclusiv cel de MySQL, sunt descoperite automat dacă fișierul lor JAR se află în classpath. Acest mecanism (Service Provider Interface) se bazează pe **încărcarea clasei driver-ului în class loader (Opțiunea B)**.
2. **Încărcarea explicită cu Class.forName()**: Metoda clasică, `Class.forName("com.mysql.cj.jdbc.Driver");`, forțează **încărcarea clasei driver-ului în class loader (Opțiunea B)**. Blocul static al clasei driver-ului se auto-înregistrează la DriverManager.
3. **Proprietatea de sistem jdbc.drivers**: Se poate specifica driver-ul pe **linia de comandă (Opțiunea A)**, folosind `java -Djdbc.drivers=com.mysql.cj.jdbc.Driver ....` DriverManager va citi această proprietate și va folosi `Class.forName()` pentru a încărca driver-ul, ceea ce ne duce tot la mecanismul de la punctul 2.

Acum să analizăm Opțiunea C: registering a Driver instance with the DriverManager.

Aceasta se referă la apelarea directă a metodei `DriverManager.registerDriver(new com.mysql.cj.jdbc.Driver());`.

Aici este subtilitatea:

- Când codul execută `new com.mysql.cj.jdbc.Driver()`, clasa driver-ului este **încărcată** de către class loader.
- Abia apoi, instanța deja creată și încărcată în memorie este pasată metodei `registerDriver()`, care doar o adaugă la lista de drivere cunoscute.

Prin urmare, metoda `registerDriver()` în sine nu realizează **încărcarea** clasei (loading); ea doar **înregistrează** o instanță care a fost deja încărcată în prealabil. Astfel, driver-ul nu este "încărcat *prin intermediul*" înregistrării, ci este înregistrat după ce a fost încărcat. Această distincție tehnică face ca opțiunea C să fie răspunsul corect.

Question 19

A user can search for books stored in the database through a web interface. The actual query should be implemented as a:

Variante de răspuns

- A. PreparedStatement
- B. Statement
- C. CallableStatement

Răspuns corect

A. PreparedStatement

Explicație

Această întrebare abordează o problemă fundamentală de securitate și eficiență în programarea bazelor de date, în special în contextul aplicațiilor web.

- **Scenariul:** Un utilizator introduce date (termeni de căutare) într-o interfață web. Aceste date trebuie folosite în siguranță într-o interogare SQL. Principala amenințare aici este **atacul de tip SQL Injection**.
- **Statement (Opțiunea B):** Un obiect Statement execută o interogare SQL statică. Pentru a include datele utilizatorului, programatorul ar trebui să construiască interogarea prin concatenare de string-uri (ex: "SELECT * FROM books WHERE title = " + userInput + " "). Această practică este **extrem de periculoasă**, deoarece un utilizator rău-intenționat ar putea introduce cod SQL în câmpul de căutare, manipulând astfel interogarea și compromițând baza de date. Prin urmare, Statement este alegerea greșită.
- **PreparedStatement (Opțiunea A):** Acesta este special conceput pentru a executa interogări SQL parametrizate. Interogarea este pre-compilată cu "înlocuitori" (placeholders), de obicei sub forma unui semn de întrebare (?). Datele de la utilizator sunt apoi adăugate în comandă folosind metode specifice (setString(), setInt(), etc.), iar driver-ul bazei de date se asigură că aceste date sunt tratate ca text literal, și nu ca parte executabilă a comenzii SQL. Acest mecanism **previne eficient atacurile de tip SQL Injection**. De asemenea, poate oferi un spor de performanță, deoarece interogarea este pre-compilată de baza de date. Este considerat **cea mai bună practică (best practice)** pentru acest scenariu.
- **CallableStatement (Opțiunea C):** Este o interfață specializată folosită pentru a executa **proceduri stocate (stored procedures)** în baza de date. Deși căutarea ar putea fi implementată ca o procedură stocată, PreparedStatement este instrumentul standard și direct pentru executarea interogărilor SQL parametrizate simple, cum este cea de căutare.

Data fiind nevoia critică de a preveni SQL Injection într-o aplicație web, **PreparedStatement** este singura alegere corectă și sigură.

Question 20

In the sequence

```
cStmt = conn.prepareCall("call ismtest(?)");
```

the "?" symbol can represent:

Variante de răspuns

- A. only an input parameter
- B. only an output parameter
- C. both an input and an output parameter

Răspuns corect

C. both an input and an output parameter

Explicație

Comanda `conn.prepareCall()` este specifică în JDBC pentru a crea un obiect de tip **CallableStatement**, care este folosit pentru a apela **proceduri stocate (stored procedures)** în baza de date.

Simbolul ? într-un **CallableStatement** este un placeholder pentru un parametru al procedurii stocate. O caracteristică esențială a procedurilor stocate este că parametrii lor pot fi de mai multe tipuri:

1. **IN (Input):** Un parametru de intrare, prin care aplicația trimite o valoare către procedura stocată. Acesta se setează folosind o metodă `setXXX()` (de ex., `cStmt.setString(1, "valoare")`).
2. **OUT (Output):** Un parametru de ieșire, prin care procedura stocată returnează o valoare către aplicație. Acesta trebuie "înregistrat" înainte de execuție, specificând tipul de date așteptat, folosind `cStmt.registerOutParameter(1, java.sql.Types.INTEGER)`.
3. **INOUT (Input/Output):** Un parametru care funcționează în ambele sensuri. Aplicația trimite o valoare inițială, procedura o poate modifica, iar aplicația poate citi valoarea finală. Acest tip de parametru necesită atât setarea valorii inițiale (`setXXX()`), cât și înregistrarea ca parametru de ieșire (`registerOutParameter()`).

Deoarece simbolul ? poate reprezenta oricare dintre aceste trei tipuri de parametri, el nu este limitat la a fi doar de intrare sau doar de ieșire. El poate fi **și de intrare, și de ieșire**,

acoperind astfel toate posibilitățile. Prin urmare, opțiunea C este cea mai cuprinzătoare și corectă.

Question 21

Considering that "rsmd" is a ResultSetMetaData object, what does the following statement return:

```
String catalog = rsmd.getCatalogName(1);
```

Variante de răspuns

- A. the table name
- B. the database name
- C. the schema name

Răspuns corect

B. the database name

Explicație

Un obiect **ResultSetMetaData** este folosit în JDBC pentru a obține informații *despre structura* unui ResultSet (adică metadata), cum ar fi numărul de coloane, numele și tipurile acestora, precum și informații despre sursa lor.

Metodele din ResultSetMetaData ne ajută să identificăm originea unei coloane. Ierarhia standard în SQL este: **Catalog → Schema → Tabelă → Coloană**.

- **getCatalogName(int column)**: Această metodă returnează numele **catalogului** din care provine coloana specificată. În majoritatea sistemelor de management al bazelor de date populare (precum MySQL, PostgreSQL, SQL Server), conceptul de "catalog" este echivalent cu cel de "**bază de date**" (**database**). Prin urmare, acest apel returnează numele bazei de date.

Pentru a nu se confunda cu celelalte opțiuni:

- Pentru a obține **numele tablei (Opțiunea A)**, s-ar folosi metoda rsmd.getTableName(1).
- Pentru a obține **numele schemei (Opțiunea C)**, s-ar folosi metoda rsmd.getSchemaName(1).

Question 22

What does the integer array produced by the following statement contain:

```
int[] results = stmt.executeBatch();
```

Variante de răspuns

- A. return codes for all executed operations
- B. the number of affected rows for each operation
- C. -1 for an operation if it did not succeed

Răspuns corect

B. the number of affected rows for each operation

Explicație

Metoda **executeBatch()** din JDBC este folosită pentru a trimite un lot (batch) de comenzi SQL (adunate anterior cu `addBatch()`) către baza de date pentru a fi executate împreună. Aceasta este o tehnică de optimizare a performanței.

Metoda returnează un tablou de numere întregi (`int[]`), în care fiecare element corespunde unei comenzi din lot, în ordinea în care au fost adăugate:

- **Pentru comenzi DML (Data Manipulation Language) precum INSERT, UPDATE sau DELETE care se execută cu succes**, valoarea din tablou este un număr întreg ce reprezintă **numărul de rânduri care au fost afectate** de comanda respectivă. Aceasta este definiția principală a conținutului tabloului, ceea ce face **Opțiunea B** corectă.
- **Opțiunea A** ("return codes") este un termen prea vag. Deși numărul de rânduri afectate poate fi considerat un "cod de retur", opțiunea B este mult mai specifică și mai corectă din punct de vedere tehnic.
- **Opțiunea C** este incorectă. Conform specificației JDBC, dacă o comandă dintr-un lot eșuează, se aruncă o excepție de tip `BatchUpdateException`. Valoarea asociată unei comenzi eșuate este constanta `Statement.EXECUTE_FAILED`, care are valoarea **-3**, nu -1.

Question 23

Given the following code sequence, what will happen if the file cannot be opened:

Java

```
try {  
    conn = DriverManager.getConnection(url, user, password);  
    BufferedReader reader = new BufferedReader(new FileReader("path/to/file.csv"));  
    //read from file and write to database  
} catch (Exception ex) {
```

```
    Logger.getLogger(Main.class.getName()).log(Level.SEVERE, null, ex);
}

if (conn!=null)

    conn.close();
```

Variante de răspuns

- A. an exception will be thrown, but the connection to the database will be created and remain open
- B. an exception will be thrown, but the connection to the database will be created and be closed
- C. the connection will never be created

Răspuns corect

B. an exception will be thrown, but the connection to the database will be created and be closed

Explicație

Să urmărim execuția codului pas cu pas, în scenariul în care fișierul nu poate fi deschis:

1. **Începe blocul try.**
2. Prima linie, `conn = DriverManager.getConnection(url, user, password);`, se execută. Presupunând că datele de conectare sunt corecte și baza de date este accesibilă, această linie **reușește**. Variabila `conn` va conține acum o conexiune validă și deschisă la baza de date.
3. A doua linie, `new BufferedReader(new FileReader("path/to/file.csv"));`, se execută. Deoarece fișierul nu poate fi deschis, `new FileReader(...)` va arunca o excepție (de exemplu, `FileNotFoundException`).
4. În momentul în care excepția este aruncată, execuția normală a blocului `try` **se oprește brusc**. Controlul este transferat imediat la blocul `catch`.
5. **Blocul catch** se execută. Excepția este prinsă și înregistrată în log.
6. După ce blocul `try-catch` s-a încheiat, programul continuă cu instrucțiunile următoare.
7. Se ajunge la instrucțiunea `if (conn!=null)`. Deoarece conexiunea a fost creată cu succes la pasul 2, `conn` **nu** este null. Condiția este adevărată.
8. Prin urmare, se execută `conn.close();`, iar conexiunea la baza de date este **închisă**.

Concluzie:

- Se aruncă o excepție.
- Conexiunea la baza de date se creează (înainte de excepție).
- Conexiunea la baza de date se închide (în blocul if de după try-catch).

Acest lanț de evenimente corespunde exact descrierii din **Opțiunea B**.

(Notă: Deși acest cod funcționează în scenariul dat, practica modernă recomandă folosirea unui bloc finally sau, și mai bine, a unei instrucțiuni try-with-resources pentru a garanta închiderea resurselor în toate situațiile posibile.)

Question 24

A CallableStatement object is used for:

Variante de răspuns

- A. the execution of a stored procedure
- B. the execution of a database function
- C. the execution of a database trigger

Răspuns corect

A. the execution of a stored procedure

Explicație

În API-ul JDBC (Java Database Connectivity), fiecare tip de obiect Statement are un rol bine definit:

- **Statement:** Pentru comenzi SQL simple, statice.
- **PreparedStatement:** Pentru comenzi SQL parametrizate, pre-compile (cea mai comună și sigură opțiune pentru SELECT, INSERT, UPDATE, DELETE).
- **CallableStatement:** Aceasta este o interfață specializată care extinde PreparedStatement. Scopul său principal și definitoriu este de a oferi o metodă standard pentru a apela **proceduri stocate (stored procedures)** din baza de date.

CallableStatement introduce o sintaxă specifică ({call nume_procedura(...)}) și metode suplimentare (precum registerOutParameter()) necesare pentru a lucra cu parametrii de ieșire (OUT) și de intrare/ieșire (INOUT), care sunt o caracteristică comună a procedurilor stocate.

- **Opțiunea B (funcție):** Deși CallableStatement poate fi folosit și pentru a apela funcții stocate (care de obicei returnează o singură valoare), rolul său principal și cel mai reprezentativ este legat de proceduri stocate.
- **Opțiunea C (trigger):** Este incorectă. Un **trigger** este un bloc de cod care este executat **automat** de către baza de date ca răspuns la o acțiune specifică (de ex., un INSERT sau UPDATE pe o tabelă). Triggerele nu pot fi apelate direct dintr-o aplicație; ele sunt invocate implicit de serverul de baze de date.

Prin urmare, utilizarea canonică a unui CallableStatement este pentru execuția procedurilor stocate.

Question 25

Which of the following is true in the relational model:

Variante de răspuns

- A. rows can have different numbers of values
- B. attribute values for each row can have different data types
- C. rows can contain undefined values

Răspuns corect

C. rows can contain undefined values

Explicație

Această întrebare testează cunoștințele despre principiile fundamentale ale modelului relațional, care stă la baza majorității bazelor de date moderne (SQL).

- **Opțiunea A: rows can have different numbers of values (rândurile pot avea un număr diferit de valori)**
 - **Fals.** O regulă de bază a modelului relațional este că o tabelă (relație) are o structură fixă, definită de un set de coloane (atribute). Fiecare rând (tuplu) din acea tabelă trebuie să se conformeze acestei structuri, ceea ce înseamnă că fiecare rând trebuie să aibă exact același număr de valori, corespunzător numărului de coloane.
- **Opțiunea B: attribute values for each row can have different data types (valorile atributelor pentru fiecare rând pot avea tipuri de date diferite)**
 - **Fals.** Fiecare coloană (atribut) dintr-o tabelă este definită cu un tip de date specific (de ex., INTEGER, VARCHAR, DATE). Toate valorile din acea coloană, indiferent de rând, trebuie să respecte acel tip de date. Nu se

poate stoca un număr într-o celulă și un text în alta, în cadrul aceleiași coloane.

- **Opțiunea C: rows can contain undefined values (rândurile pot conține valori nedefinite)**
 - **Adevărat.** Acesta este exact scopul pentru care a fost introdus marcajul special **NULL** în SQL. NULL nu este o valoare (precum 0 sau un string gol), ci un indicator că valoarea pentru acea celulă este **nedefinită**, necunoscută sau neaplicabilă. Un rând poate conține NULL în una sau mai multe coloane, cu condiția ca acele coloane să nu aibă constrângerea NOT NULL.

Prin urmare, singura afirmație adevărată despre modelul relațional dintre cele trei este C.

Se pare că aceasta este aceeași întrebare ca cea anterioară. Voi oferi din nou analiza detaliată.

Question 26

Which of the following is true in the relational model:

Variante de răspuns

- A. rows can have different numbers of values
- B. attribute values for each row can have different data types
- C. rows can contain undefined values

Răspuns corect

C. rows can contain undefined values

Explicație

Această întrebare se referă la regulile fundamentale ale modelului relațional, care stă la baza bazelor de date SQL.

- **Opțiunea A: rows can have different numbers of values (rândurile pot avea un număr diferit de valori)**
 - **Fals.** O regulă de bază a modelului relațional este că o tabelă (relație) are o structură fixă, definită de un set de coloane (atribute). Fiecare rând (tuplu) din acea tabelă trebuie să se conformeze acestei structuri, ceea ce înseamnă că fiecare rând trebuie să aibă exact același număr de valori, corespunzător numărului de coloane.

- **Opțiunea B: attribute values for each row can have different data types (valorile atributelor pentru fiecare rând pot avea tipuri de date diferite)**
 - **Fals.** Fiecare coloană (atribut) dintr-o tabelă este definită cu un tip de date specific (de ex., INTEGER, VARCHAR, DATE). Toate valorile din acea coloană, indiferent de rând, trebuie să respecte acel tip de date. Nu se poate stoca un număr într-o celulă și un text în alta, în cadrul aceleiași coloane.
- **Opțiunea C: rows can contain undefined values (rândurile pot conține valori nedefinite)**
 - **Adevărat.** Acesta este exact scopul pentru care a fost introdus marcajul special **NULL** în SQL. NULL nu este o valoare (precum 0 sau un string gol), ci un indicator că valoarea pentru acea celulă este **nedefinită**, necunoscută sau neaplicabilă. Un rând poate conține NULL în una sau mai multe coloane, cu condiția ca acele coloane să nu aibă constrângerea NOT NULL.

Prin urmare, singura afirmație adevărată despre modelul relațional dintre cele trei este C.

Question 27

Primary keys

Variante de răspuns

- A. Uniquely identify a tuple
- B. Can have undefined values
- C. Can have duplicate values

Răspuns corect

A. Uniquely identify a tuple

Explicație

Această întrebare se referă la proprietățile fundamentale ale unei **chei primare (Primary Key)** în modelul relațional. O cheie primară este o constrângere specială aplicată uneia sau mai multor coloane dintr-o tabelă și are două reguli esențiale, cunoscute împreună ca **regula integrității entității (entity integrity)**:

1. **Unicitate (Uniqueness):** Valoarea cheii primare trebuie să fie unică pentru fiecare rând (în termeni formali, **tuplu**) din tabelă. Scopul său principal este de a servi ca un identificator unic și fără ambiguități pentru fiecare înregistrare. Aceasta face ca **Opțiunea A** să fie **corectă**.

2. **Non-nulitate (Not Null):** O cheie primară nu poate conține valori nule (NULL). NULL reprezintă o valoare necunoscută sau **nedefinită**, iar un identificator nu poate fi necunoscut. Fiecare rând trebuie să aibă o valoare reală și definită pentru cheia sa primară. Aceasta face ca **Opțiunea B** (Can have undefined values) să fie **incorectă**.

Din prima regulă (unicitate) reiese direct că o cheie primară nu poate avea valori duplicate, ceea ce face ca și **Opțiunea C** (Can have duplicate values) să fie **incorectă**.

Prin urmare, singura afirmație adevărată și care descrie funcția principală a unei chei primare este A.

Question 28

What does the column encpass contain after an insert, given the following trigger:

SQL

```
CREATE TRIGGER upd_enc BEFORE INSERT ON ism_passwords FOR EACH ROW SET  
NEW.encpass = encpassword(NEW.encpass);
```

Variante de răspuns

- A. the value contained in encpass
- B. the value returned by the function encpassword
- C. an insert would raise an error

Răspuns corect

B. the value returned by the function encpassword

Explicație

Să analizăm în detaliu ce face trigger-ul:

1. **CREATE TRIGGER upd_enc:** Definește un trigger cu numele upd_enc.
2. **BEFORE INSERT ON ism_passwords:** Specifică momentul când se declanșează trigger-ul: **înainte** ca o nouă înregistrare să fie efectiv inserată în tabela ism_passwords.
3. **FOR EACH ROW:** Indică faptul că acțiunea trigger-ului se va executa pentru fiecare rând în parte care este inserat.
4. **SET NEW.encpass = encpassword(NEW.encpass);:** Aceasta este acțiunea propriu-zisă a trigger-ului.
 - În contextul unui trigger INSERT, NEW este un cuvânt cheie special care se referă la rândul ce urmează a fi inserat.

- NEW.encpass se referă la valoarea care a fost furnizată pentru coloana encpass în comanda INSERT.
- Trigger-ul ia această valoare inițială, o pasează ca argument funcției encpassword(), iar apoi **suprascrie** valoarea din NEW.encpass cu rezultatul returnat de funcție.

Deoarece trigger-ul se execută **înainte** de inserare și **modifică** valoarea coloanei encpass, valoarea care va fi în final stocată în tabelă nu este cea originală, ci cea procesată, adică **valoarea returnată de funcția encpassword**. Aceasta este o tehnică standard pentru a cripta sau a "hăshui" parolele automat, înainte de a le salva în baza de date.

Question 29

Given the following code

```
ResultSet rs = stmt.executeQuery("SELECT COUNT(*) FROM employees");
```

which is not a way to retrieve the total number of records from the result set:

Variante de răspuns

- A. rs.last()
- B. rs.getInt(0)
- C. rs.getInt("COUNT(*)")

Răspuns corect

B. rs.getInt(0)

Explicație

Să analizăm comanda și opțiunile:

1. **Comanda SQL:** SELECT COUNT(*) FROM employees returnează un ResultSet care conține **un singur rând** și **o singură coloană**. Valoarea din acea unică celulă este numărul total de înregistrări din tabela employees.
2. **Accesarea datelor:** Pentru a accesa orice dată dintr-un ResultSet, cursorul trebuie mai întâi mutat pe un rând valid (de ex., apelând rs.next()). Presupunând că acest pas a fost făcut, analizăm opțiunile de extragere a valorii:
 - **Opțiunea B: rs.getInt(0)**
 - **Incorrect.** Aceasta este cea mai clară greșeală. În JDBC, indexarea coloanelor într-un ResultSet este **1-based**, nu 0-based. Prima coloană se află la indexul 1, a doua la indexul 2, și așa mai departe. Un apel la

rs.getInt(0) va încerca să acceseze o coloană inexistentă și va arunca întotdeauna o excepție SQLException. Prin urmare, aceasta **nu** este o metodă validă de a extrage date.

- **Opțiunea C: rs.getInt("COUNT(*)")**
 - **Corect.** Aceasta este o metodă validă de a extrage valoarea. Se accesează coloana după numele ei. Multe baze de date (inclusiv MySQL) numesc coloana rezultată dintr-un COUNT(*) exact "COUNT(*)". Alternativ, s-ar fi putut folosi indexul corect: rs.getInt(1).
- **Opțiunea A: rs.last()**
 - Aceasta este o metodă de **navigare**, nu de extragere a datelor. rs.last() mută cursorul pe ultimul rând din ResultSet (ceea ce, în cazul nostru, este același cu primul și singurul rând). Metoda în sine returnează un boolean și nu valoarea din celulă. Deși nu extrage valoarea, este un pas valid în procesul de navigare. Însă, rs.getInt(0) este o încercare de extragere fundamental greșită.

Prin urmare, rs.getInt(0) este singura variantă care nu poate funcționa niciodată pentru a extrage date dintr-un ResultSet din cauza regulii de indexare de la 1.

Question 30

When an entry in the LDIF has a multi value attribute, values are written

Variante de răspuns

- A. All values on a single line separated by whitespace
- B. One line for all values separated by double whitespaces
- C. One line for each value
- D. In a separate file LDIFMV(LDIF for multi value attributes)

Răspuns corect

C. One line for each value

Explicație

LDIF (LDAP Data Interchange Format) este un format standardizat pentru a reprezenta conținutul și modificările dintr-un director LDAP (Lightweight Directory Access Protocol).

Sintaxa LDIF pentru atribut este <nume-atribut>: <valoare>.

Atunci când un atribut are mai multe valori (este "multi-valued"), standardul LDIF specifică faptul că linia nume-atribut: valoare se repetă pentru fiecare valoare în parte. Fiecare valoare este scrisă pe propria sa linie, prefixată de numele atributului.

Exemplu clasic: Atributul objectClass este aproape întotdeauna multi-valorat. O intrare LDIF ar arăta astfel:

LDIF

dn: uid=test,ou=users,dc=example,dc=com

objectClass: top

objectClass: person

objectClass: organizationalPerson

objectClass: inetOrgPerson

cn: Test User

sn: User

După cum se observă, fiecare valoare pentru objectClass este pe o linie separată.

- Opțiunile A și B sunt incorecte, deoarece încalcă sintaxa de bază a LDIF.
- Opțiunea D este incorectă, deoarece nu există un astfel de standard sau format de fișier separat numit LDIFMV.

Question 31

While using JDBC a parametrized statement can be created with the method:

Variante de răspuns

- A. prepareCall
- B. createStatement
- C. prepareStatement
- D. executeStatement

Răspuns corect

C. prepareStatement

Explicație

În JDBC, interfața standard pentru lucrul cu comenzi SQL parametrizate (care folosesc ? ca placeholder pentru valori) este **PreparedStatement**.

Pentru a crea un astfel de obiect, se folosește o metodă a obiectului Connection. Să analizăm opțiunile:

- **prepareStatement(String sql) (Opțiunea C):** Aceasta este metoda corectă. Se apelează pe un obiect Connection și returnează un obiect PreparedStatement pre-compilat, gata să primească valori pentru parametrii săi.
- **prepareCall(String sql) (Opțiunea A):** Această metodă este, de asemenea, pentru o comandă parametrizată, dar este specifică pentru crearea unui obiect CallableStatement, folosit exclusiv pentru a apela **proceduri stocate**. Deși este un tip de "parametrized statement", prepareStatement este răspunsul general și corect pentru o comandă SQL parametrizată standard.
- **createStatement() (Opțiunea B):** Această metodă creează un obiect Statement simplu, care este folosit pentru a executa comenzi SQL statice, **fără** parametri.
- **executeStatement (Opțiunea D):** Acesta nu este un nume de metodă valid în API-ul JDBC pentru crearea unui statement. Metodele de execuție (executeQuery, executeUpdate) se găsesc pe obiectele Statement, nu pe Connection.

Prin urmare, metoda standard și corectă pentru a crea o comandă parametrizată este **prepareStatement**.

Întrebarea 33

Question: One privilege is

Variante de răspuns:

- A. a not a right to execute a SQL command over an object from the schema
- B. a right to execute a SQL command over an object from the schema
- C. a right to execute a SQL command over an object from a different schema
- D. not a right to execute a SQL command over an object from a different schema

Răspuns corect

B. a right to execute a SQL command over an object from the schema

Explicație

Un **privilegiu**, în contextul securității bazelor de date, reprezintă **dreptul** acordat unui utilizator de a executa o anumită operațiune (de obicei, o comandă SQL) asupra unui obiect specific din baza de date.

Să analizăm opțiunile:

- **Opțiunea B** descrie cel mai bine această definiție fundamentală. Un privilegiu este un **drept** (a right) de a rula o **comandă SQL** (to execute a SQL command) asupra unui **obiect** (over an object - de ex., o tabelă, o vizualizare, o funcție) care aparține unei scheme.
- **Opțiunile A și D** sunt incorecte deoarece neagă faptul că un privilegiu este un drept ("not a right").
- **Opțiunea C** este prea specifică și, prin urmare, incorectă ca definiție generală. Deși un utilizator poate primi un privilegiu asupra unui obiect dintr-o altă schemă, aceasta este doar o aplicare a conceptului. Definiția de bază a privilegiului nu depinde de locația obiectului (în aceeași schemă sau într-una diferită). Opțiunea B este mai generală și fundamental corectă.

Întrebarea 34

Question: In a LDIF file for an entry, DN contains

Variante de răspuns:

- A. none is correct
- B. only uid of the entry which is unique in the Directory Services
- C. only cn of the entry which is unique in the Directory Services
- D. the entire address of the entry which is unique in the Directory Services

Răspuns corect

D. the entire address of the a entry which is unique in the Directory Services

Explicație

Întrebarea se referă la rolul și structura **DN-ului (Distinguished Name)** într-o intrare LDIF (LDAP Data Interchange Format).

DN-ul (Distinguished Name) este identificatorul unic absolut pentru o intrare într-un director LDAP. Funcționează similar cu o cale completă către un fișier într-un sistem de fișiere (de exemplu, C:\Users\John\Documents\file.txt).

- Un DN este format dintr-o secvență de RDN-uri (Relative Distinguished Names), care împreună formează o cale ierarhică de la rădăcina directorului până la intrarea specifică.
- Un exemplu de DN este: cn=Ion Popescu,ou=Marketing,dc=companie,dc=ro.

Să analizăm opțiunile:

- **Opțiunile B și C** sunt incorecte. uid (user id) sau cn (common name) sunt de obicei doar prima componentă a DN-ului (RDN-ul), nu DN-ul întreg. De exemplu, cn=Ion Popescu este RDN-ul, nu DN-ul complet. DN-ul conține întreaga cale.
- **Opțiunea D** este cea mai corectă descriere. DN-ul reprezintă "**adresa întreagă**" sau calea completă a intrării, iar proprietatea sa fundamentală este că trebuie să fie **unică** în întregul serviciu de director (Directory Services).

Întrebarea 35

Question: For what attributes the following reference is used ":/"

Variante de răspuns:

- A. for clone attributes
- B. for binary attributes
- C. for files attributes
- D. for protoattributes

Răspuns corect

C. for files attributes

Explicație

Întrebarea, deși formulată neclar (":/"), se referă cel mai probabil la o sintaxă specifică din formatul **LDIF (LDAP Data Interchange Format)** pentru a include valori ale atributelor dintr-o sursă externă.

Sintaxa corectă și standard în LDIF este:

nume-atribut: < URL

Unde:

- Caracterul < indică faptul că valoarea atributului nu este textul care urmează, ci trebuie citită din locația specificată de **URL**.
- Cel mai adesea, acest URL este de tip file:/// , indicând o cale către un fișier local. De exemplu: jpegPhoto: < file:///home/user/poza_profil.jpg.

Acest mecanism este utilizat în mod special pentru attributele a căror valoare este conținutul unui **fișier**, cum ar fi o imagine (jpegPhoto), un certificat digital (userCertificate;binary), sau orice alt tip de document.

Analizând opțiunile:

- **Opțiunea C (for files attributes - pentru attribute de tip fișier)** descrie perfect scopul acestei sintaxe.
- Opțiunea B (pentru attribute binare) este parțial adevărată, deoarece fișierul referit poate fi binar, dar sintaxa se referă la referirea fișierului în sine, nu exclusiv la conținutul binar. Pentru conținut binar se poate folosi și codarea Base64 (cu sintaxa ::).
- Opțiunile A și D folosesc termeni care nu sunt standard în contextul LDAP/LDIF.

Întrebarea 37

Question: The orcladmin user selected is

Variante de răspuns:

- A. the superuser of the installed OID
- B. is a fresh user created in the realm
- C. none is correct
- D. is a simple user in the realm

Răspuns corect

A. the superuser of the installed OID

Explicație

Utilizatorul **orcladmin** este contul administrativ implicit creat în timpul procesului de instalare și configurare a **Oracle Internet Directory (OID)**, serviciul de director LDAP al Oracle.

- Acest utilizator are cel mai înalt nivel de privilegii în cadrul directorului și este considerat **superuser-ul** sau administratorul principal al instanței OID. Rolul său este esențial pentru gestionarea inițială și continuă a directorului, inclusiv crearea altor utilizatori, definirea politicilor de securitate și administrarea schemei.
- Screenshot-ul din interfața Oracle Directory Manager confirmă acest lucru. Pentru intrarea orcladmin selectată, atributul **description** are valoarea "**Super Administrator for subscriber**", ceea ce indică în mod clar rolul său privilegiat.

Analizând opțiunile:

- **Opțiunea A** este corectă, definind precis rolul de superuser.

- **Opțiunile B și D** sunt incorecte, deoarece orcladmin nu este un utilizator simplu sau nou-creat ("fresh user") în sensul unui cont standard, ci este contul administrativ principal.
- **Opțiunea C** este incorectă, deoarece opțiunea A este validă.

Întrebarea 39

Question: In a LDIF file, the lines in an entry continues on the next row using

Variante de răspuns:

- A. ENTER
- B. TAB
- C. SHIFT
- D. CTRL

Răspuns corect

Niciuna dintre opțiuni nu este complet corectă, dar răspunsul intenționat este cel mai probabil **A. ENTER**, cu o explicație importantă.

Explicație

Standardul **LDIF (LDAP Data Interchange Format)** are o regulă specifică pentru continuarea unei linii lungi pe rândul următor, un proces numit "**line folding**" (împăturirea liniei).

Regula corectă este:

O linie logică poate fi împărțită în mai multe linii fizice prin introducerea unei secvențe de sfârșit de linie (newline), urmată imediat de un singur caracter spațiu (' ').

Exemplu:

O linie lungă precum:

description: This is a very long description that needs to be wrapped onto multiple lines.

Poate fi scrisă în LDIF astfel:

LDIF

description: This is a very long description that needs to be wrapped on
to multiple lines.

Când un program citește acest fișier, va uni cele două linii eliminând secvența \n (newline + spațiu).

Analiza opțiunilor:

Opțiunile sunt formulate foarte prost, referindu-se la taste, nu la caracterele din fișier.

- **C (SHIFT)** și **D (CTRL)** sunt taste modificatoare și nu introduc caractere, deci sunt incorecte.
- **B (TAB)** este incorect; caracterul TAB nu are niciun rol în continuarea liniilor.
- **A (ENTER)** este tasta care, în general, introduce un caracter **newline** (sfârșit de linie).

Deși apăsarea tastei ENTER este doar prima parte a procesului (deoarece este necesar și un spațiu la începutul liniei următoare), este singura opțiune care are legătură cu mecanismul real. Prin urmare, deși este un răspuns incomplet și tehnic imprecis, este singurul plauzibil dintre cele date.

Întrebarea 40

Question: Which users are installed directly with OID in cn=Users

Variante de răspuns:

- A. orcladmin and PUBLIC
- B. orcladmin, PUBLIC and PRIVATE, which is deleted in this installation
- C. none are correct
- D. orcladmin, oidadmin users

Răspuns corect

A. orcladmin and PUBLIC

Explicație

Întrebarea se poate răspunde direct prin examinarea screenshot-ului furnizat din **Oracle Directory Manager**.

1. În panoul de navigare din stânga, se poate observa ierarhia directorului Oracle Internet Directory (OID).
2. Urmărind calea cn=OracleContext -> cn=Products -> cn=OracleContext, ajungem la containerul **cn=Users**.
3. Direct sub acest container, cn=Users, sunt vizibile două intrări (care reprezintă utilizatori sau entități similare):
 - **cn=orcladmin**
 - **cn=PUBLIC**

Acestea sunt cele două entități standard care sunt create implicit în containerul cn=Users la instalarea OID.

- **orcladmin** este contul de super-administrator.
- **PUBLIC** este o entitate specială utilizată în politicile de control al accesului pentru operațiuni neautentificate sau publice.

Prin urmare, opțiunea A corespunde exact cu ceea ce este vizibil în interfața de administrare și cu structura standard a unei instalări OID. Celelalte opțiuni introduc nume de utilizatori fictive (PRIVATE, oidadmin) care nu fac parte din instalarea implicită.

Întrebarea 42

Question: Given a table with the structure: students(student_id, student_name, course_name) can anomalies appear on an update to student_name?

Variante de răspuns:

- A. Yes
- B. No

Răspuns corect

A. Yes

Explicație

Da, anomaliiile de actualizare (update anomalies) pot apărea. Să analizăm de ce.

1. **Structura și Redundanța Datelor:** În tabela students(student_id, student_name, course_name), se presupune că un student se poate înscrie la mai multe cursuri. Acest lucru duce la repetarea datelor studentului pentru fiecare curs. De exemplu:

student_id	student_name	course_name
101	Ana Popescu	Matematică
101	Ana Popescu	Istorie
102	Ion Ionescu	Fizică

Numele "Ana Popescu" este stocat de două ori, o dată pentru fiecare curs la care este înscrisă. Aceasta este o ****redundanță a datelor****.

2. **Anomalia de Actualizare (Update Anomaly):** O anomalie de actualizare apare atunci când modificarea unei singure informații logice necesită actualizarea mai multor înregistrări în baza de date.

- Să presupunem că Ana Popescu se căsătorește și își schimbă numele în Ana Vasilescu.
- Pentru a reflecta corect această schimbare, trebuie să actualizăm **toate** înregistrările din tabelă unde apare numele ei.
- Dacă am actualiza doar o singură înregistrare, baza de date ar deveni inconsistentă:

student_id	student_name	course_name
101	Ana Vasilescu	Matematică
101	Ana Popescu	Istorie
102	Ion Ionescu	Fizică

Acum, baza de date ar oferi două nume diferite pentru același student (cu `student\_id` 101), ceea ce este o eroare. Această potențială inconsistență este exact o anomalie de actualizare. Problema apare din cauza faptului că tabela nu este normalizată corespunzător (încalcă Forma Normală 2 și 3).

Întrebarea 43

Question: Which of the following statements is true?

Variante de răspuns:

- A. In FN2 there are no transitive dependencies
- B. In FN1 redundant fields are accepted
- C. In FN3 there are no partial dependencies
- D. In FN3 transitive dependencies are accepted

Răspuns corect

C. In FN3 there are no partial dependencies

(Notă: Această întrebare este formulată ambiguu, deoarece atât opțiunea B, cât și opțiunea C sunt tehnic adevărate. Totuși, opțiunea C este cea mai riguroasă și probabil răspunsul intenționat.)

Explicație

Să analizăm fiecare afirmație pe baza ierarhiei formelor normale (FN):

- **A. In FN2 there are no transitive dependencies (În FN2 nu există dependențe tranzitive)**
 - **Fals.** O tabelă poate fi în Forma Normală 2 (FN2) și totuși să conțină dependențe tranzitive. Eliminarea dependențelor tranzitive este obiectivul Formei Normale 3 (FN3).
- **B. In FN1 redundant fields are accepted (În FN1 sunt acceptate câmpuri redundante)**
 - **Adevărat.** Prima Formă Normală (FN1) se ocupă doar de eliminarea grupurilor repetitive și de asigurarea atomicității datelor. Ea nu adresează redundanța datelor care apare din cauza dependențelor parțiale sau tranzitive. Această redundanță este "acceptată" și este eliminată progresiv în FN2 și FN3.
- **C. In FN3 there are no partial dependencies (În FN3 nu există dependențe parțiale)**
 - **Adevărat.** Regulile formelor normale sunt cumulative. Pentru ca o tabelă să fie în FN3, ea trebuie mai întâi să fie în FN2. Definiția FN2 impune eliminarea tuturor dependențelor parțiale. Prin urmare, dacă o tabelă este în FN3, este garantat că nu are dependențe parțiale.
- **D. In FN3 transitive dependencies are accepted (În FN3 sunt acceptate dependențe tranzitive)**
 - **Fals.** Definiția fundamentală a Formei Normale 3 este tocmai eliminarea dependențelor tranzitive dintr-o tabelă care se află deja în FN2.

Concluzie:

Deși ambele afirmații B și C sunt corecte, afirmația C este o consecință directă și definitorie a ierarhiei formelor normale și este, prin urmare, considerată un răspuns mai puternic și mai precis în contextul unui test. Ea descrie o proprietate garantată a unei tabele care a atins nivelul FN3.

Întrebarea 44

Question: What does the following code sequence return in MYSQL
`aes_decrypt(aes_encrypt(s,key),key)`

Variante de răspuns:

- A. The character string „key"

- B. A character string different from „s" or „key"
- C. An error
- D. The character string „s"

Răspuns corect

D. The character string „s"

Explicație

Funcțiile `aes_encrypt` și `aes_decrypt` din MySQL implementează algoritmul de **criptare simetrică** AES.

Principiul fundamental al criptării simetrice este că **aceeași cheie** este folosită atât pentru a cripta datele, cât și pentru a le decripta.

Să analizăm secvența:

1. **`aes_encrypt(s, key)`**: Această parte a codului ia string-ul original (s) și îl criptează folosind cheia (key). Rezultatul este un șir de date binar, ilizibil (ciphertext).
2. **`aes_decrypt(..., key)`**: Această parte ia rezultatul criptat de la pasul anterior și aplică operația de decriptare folosind **exact aceeași cheie (key)**.

Deoarece pentru decriptare se folosește aceeași cheie ca la criptare, procesul va inversa perfect operația de criptare, iar rezultatul final va fi string-ul original, adică **s**.

Acest caz este diferit de cel din întrebarea 7, unde cheile pentru criptare și decriptare erau diferite (key1 și key2), motiv pentru care rezultatul era "altceva" (date corupte). Aici, folosirea aceleiași chei garantează recuperarea datelor originale.

Întrebarea 46

Question: Which of the following is true in the relational model?

Variante de răspuns:

- A. An attribute is a column of a table
- B. An attribute is a table
- C. An attribute is a line of a table
- D. An attribute is a data type

Răspuns corect

A. An attribute is a column of a table

Explicație

Această întrebare se referă la terminologia de bază a modelului relațional, care folosește termeni formali matematici, și corespondența acestora cu termenii comuni din bazele de date SQL.

Correspondența este următoarea:

- **Relație (Relation)** în modelul formal corespunde cu o **Tabelă (Table)** în implementările SQL.
- **Tuplu (Tuple)** corespunde cu un **Rând (Row)** sau o linie/înregistrare.
- **Atribut (Attribute)** corespunde cu o **Coloană (Column)** sau un câmp.

Analizând opțiunile:

- **A. An attribute is a column of a table (Un atribut este o coloană a unei table):** Aceasta este definiția corectă, reflectând corespondența dintre modelul teoretic și implementarea practică.
- **B. An attribute is a table (Un atribut este o tabelă):** Incorect. O tabelă este o relație.
- **C. An attribute is a line of a table (Un atribut este o linie a unei table):** Incorect. O linie este un tuplu.
- **D. An attribute is a data type (Un atribut este un tip de date):** Incorect. Un atribut **are** un tip de date (mai formal, este definit pe un domeniu), dar nu **este** tipul de date în sine. De exemplu, nume_student este atributul, iar VARCHAR(50) este tipul său de date.

Întrebarea 47

Question: Which command is correct

Variante de răspuns:

- A. sqlplus /as system
- B. sqlplus /as sysdbadmin
- C. sqlplus /as sysadmin
- D. sqlplus /as sysdba

Răspuns corect

D. sqlplus /as sysdba

Explicație

Această întrebare testează sintaxa corectă pentru conectarea la o bază de date Oracle cu privilegii administrative, folosind utilitarul de linie de comandă **SQL*Plus**.

Să analizăm componentele comenzii:

- **sqlplus**: Numele programului.
- **/**: O prescurtare pentru conectarea la baza de date locală folosind autentificarea sistemului de operare. Utilizatorul care rulează comanda trebuie să facă parte dintr-un grup special al sistemului de operare (de obicei dba sau oradba pe Linux/Unix, sau ORA_DBA pe Windows).
- **as <privilegiu>**: O clauză care specifică rolul administrativ special cu care se face conexiunea.

Oracle definește câteva privilegii administrative puternice, cele mai importante pentru conectare fiind:

- **SYSDBA**: Cel mai înalt nivel de privilegiu. Acordă control total asupra instanței bazei de date, inclusiv pornirea, oprirea, crearea bazei de date și operațiuni de recuperare.
- **SYSOPER**: Un privilegiu mai limitat, care permite operațiuni de bază precum pornirea, oprirea și backup-ul, dar nu și accesul la datele utilizatorilor.

Analizând opțiunile:

- **A. as system**: Incorect. SYSTEM este un cont de utilizator administrativ, dar nu este un privilegiu de conectare.
- **B. as sysdbadmin**: Incorect. Numele privilegiului este SYSDBA.
- **C. as sysadmin**: Incorect. sysadmin este un rol în alte sisteme de baze de date (precum SQL Server), nu un privilegiu de conectare în Oracle.
- **D. as sysdba**: **Corect**. Aceasta este sintaxa standard pentru a se conecta cu privilegiul de super-administrator SYSDBA.

Întrebarea 48

Question: Which of the following commands returns all clients who have bought at least a product?

Variante de răspuns:

- A. select * from clients where id=any(select cust_id from orders)
- B. select * from clients having id=any(select cust_id from orders)
- C. select * from clients where id = select cust_id from orders

- D. select * from clients having id=(select cust_id from orders)

Răspuns corect

A. select * from clients where id=any(select cust_id from orders)

Explicație

Scopul interogării este de a găsi toți clienții al căror id se regăsește în coloana cust_id din tabela orders. Acest lucru necesită filtrarea rândurilor din tabela clients pe baza unui set de valori dintr-o altă tabelă (o subinterogare).

Să analizăm opțiunile:

1. Clauza de filtrare: WHERE vs. HAVING

- Clauza WHERE este folosită pentru a filtra rânduri individuale. Aceasta este clauza corectă pentru problema dată.
- Clauza HAVING este folosită pentru a filtra grupuri de rânduri, după ce a fost aplicată o clauză GROUP BY și o funcție de agregare. Folosirea ei aici, fără GROUP BY, este incorectă din punct de vedere sintactic și logic. Prin urmare, **opțiunile B și D sunt incorecte.**

2. Operatorul de comparație: = ANY vs. =

- Subinterogarea (select cust_id from orders) returnează o listă de valori (mai multe cust_id-uri), nu o singură valoare.
- Operatorul simplu = (folosit în opțiunea C) poate compara o valoare doar cu o altă valoare unică (scalară). Încercarea de a-l folosi pentru a compara id cu o listă de rezultate va genera o eroare. Prin urmare, **opțiunea C este incorectă.**
- Operatorul = **ANY** este un predicat cuantificat în SQL. valoare = ANY (lista) este echivalent din punct de vedere logic cu valoare IN (lista). El verifică dacă id-ul clientului este egal cu **oricare** dintre valorile returnate de subinterogare. Aceasta este exact logica necesară pentru a rezolva problema. Prin urmare, **opțiunea A este corectă.**

Întrebarea 49

Question: Which of the following is true?

Variante de răspuns:

- A. In FN2 there are redundant fields
- B. In FN2 there are transitive dependencies

- C. In FN3 there are redundant fields
- D. In FN3 there are transitive dependencies

Răspuns corect

B. In FN2 there are transitive dependencies

(Notă: Similar cu o întrebare anterioară, aceasta are o ambiguitate, deoarece și opțiunea A poate fi considerată adevărată. Totuși, opțiunea B este mai precisă din punct de vedere tehnic.)

Explicație

Să evaluăm fiecare afirmație în contextul teoriei normalizării:

- **D. In FN3 there are transitive dependencies (În FN3 există dependențe tranzitive)**
 - **Fals.** Definiția explicită a Formei Normale 3 (FN3) este că o tabelă trebuie să fie în FN2 și să **nu** aibă dependențe tranzitive.
- **C. In FN3 there are redundant fields (În FN3 există câmpuri redundante)**
 - **În general, fals.** Scopul principal al normalizării până la FN3 este tocmai eliminarea celor mai comune forme de redundanță a datelor, cauzate de dependențe parțiale și tranzitive. Deși pot exista forme mai subtile de redundanță (adresate de BCNF sau FN4), în contextul FN1-FN3, această afirmație este considerată falsă.
- **A. In FN2 there are redundant fields (În FN2 există câmpuri redundante)**
 - **Adevărat.** O tabelă în FN2 a eliminat redundanța cauzată de dependențe parțiale, dar poate încă suferi de redundanță cauzată de dependențele tranzitive. Acesta este exact motivul pentru care există FN3.
- **B. In FN2 there are transitive dependencies (În FN2 există dependențe tranzitive)**
 - **Adevărat.** O tabelă în FN2 nu are constrângeri împotriva dependențelor tranzitive. Posibilitatea existenței acestora este caracteristica ce diferențiază FN2 de FN3.

De ce este B răspunsul mai bun decât A?

Ambele afirmații A și B sunt adevărate, dar descriu aceeași problemă din perspective diferite.

- Afirmația B, "În FN2 există dependențe tranzitive", descrie **cauza** structurală, un termen tehnic precis din teoria normalizării.

- Afirmatia A, "În FN2 există câmpuri redundante", descrie **efectul** sau simptomul acestei probleme structurale.

Într-un context tehnic, identificarea tipului de dependență (cauza) este mai fundamentală și mai precisă decât observarea simptomului (redundanța). Prin urmare, **opțiunea B este considerată răspunsul cel mai corect.**

Întrebarea 50

Question: Any privilege given to PUBLIC become a privilege...

Variante de răspuns:

- A. to any user from the database
- B. to the dropped users
- C. to nobody
- D. only to sysdba and sysoper users

Răspuns corect

A. to any user from the database

Explicație

În majoritatea sistemelor de management al bazelor de date (precum Oracle, PostgreSQL), **PUBLIC** nu este un utilizator obișnuit, ci un **rol special** sau un grup.

Proprietatea fundamentală a rolului PUBLIC este că **toți utilizatorii din baza de date** (atât cei existenți, cât și cei care vor fi creați în viitor) sunt considerați membri implicați ai acestui grup.

Prin urmare, atunci când se acordă un privilegiu rolului PUBLIC (de exemplu, GRANT SELECT ON tabela_x TO PUBLIC;), acel privilegiu devine automat disponibil pentru **absolut orice utilizator** din baza de date. Este o metodă de a acorda permisiuni globale, pentru toți.

Analizând opțiunile:

- **A. to any user from the database (către orice utilizator din baza de date):** Aceasta este definiția corectă a modului în care funcționează privilegiile acordate lui PUBLIC.
- **B. to the dropped users (către utilizatorii șterși):** Incorect. Utilizatorii șterși nu mai există și nu mai au privilegii.
- **C. to nobody (către nimeni):** Incorect. Este exact opusul.

- **D. only to sysdba and sysoper users (doar către utilizatorii sysdba și sysoper):**
Incorect. Privilegiile PUBLIC se aplică tuturor, inclusiv utilizatorilor simpli, nu doar celor administrativi.

Întrebarea 53

Question: LDAP(Lightweight directory access protocol) is

Variante de răspuns:

- A. none is correct
- B. not a relational database
- C. a model of distributed database
- D. a lite relational database

Răspuns corect

B. not a relational database

Explicație

Întrebarea cere o definiție sau o clasificare a LDAP în contextul sistemelor de baze de date.

1. Modelul de date:

- **LDAP** (și serviciile de director pe care le accesează) utilizează un model de date **ierarhic**, bazat pe o structură arborescentă (Directory Information Tree - DIT). Informația este stocată în "intrări" (entries), care au atribute și valori, similar cu obiectele.
- **Bazele de date relaționale** utilizează un model **tabular**, bazat pe relații (tabele), tupluri (rânduri) și atribute (coloane).

2. Analiza opțiunilor:

- **B. not a relational database (nu este o bază de date relațională):**
Aceasta este o afirmație fundamental **adevărată**. Datorită modelului de date complet diferit (ierarhic vs. tabular), LDAP nu este și nu funcționează ca o bază de date relațională.
- **D. a lite relational database (o bază de date relațională "lite"):** Incorect. "Lightweight" (ușor) din numele protocolului se referă la faptul că este o versiune mai simplă a protocolului său predecesor (X.500 DAP), nu că ar fi o versiune "lite" a unei baze de date relaționale.

- **C. a model of distributed database (un model de bază de date distribuită):** Deși serviciile de director LDAP sunt adesea implementate ca sisteme distribuite (cu replicare și partiționare), această caracteristică descrie arhitectura, nu modelul fundamental de date. Și bazele de date relaționale pot fi distribuite. Prin urmare, afirmația de la punctul B este mult mai precisă și fundamentală pentru definirea LDAP.

Deoarece LDAP se bazează pe un model ierarhic, nu relațional, cea mai corectă și definitorie afirmație este că **nu este o bază de date relațională**.

Întrebarea 54

Question: Telephonenumber is

Variante de răspuns:

- A. mandatory attribute
- B. none is correct
- C. protoattribute
- D. optional attribute

Răspuns corect

D. optional attribute

Explicație

Pentru a determina dacă un atribut este obligatoriu (mandatory) sau opțional, trebuie să analizăm definiția **claselor de obiecte (objectClass)** specificate în intrarea LDIF.

Această intrare are următoarele clase: top, person, organizationalPerson, și inetOrgPerson.

1. **Ierarhia claselor:** inetOrgPerson este cea mai specifică clasă și moștenește attributele și regulile de la celelalte.
2. **Atribute obligatorii:** Conform schemelor standard LDAP (definite în RFC-uri), pentru ca o intrare de tip inetOrgPerson să fie validă, ea trebuie să conțină attributele obligatorii moștenite de la clasele părinte:
 - Din top: objectClass
 - Din person: sn (surname) și cn (common name)
3. **Atributul telephoneNumber:** Acest atribut este definit în clasa organizationalPerson (pe care inetOrgPerson o moștenește). În schema standard, telephoneNumber este definit ca fiind un atribut **opțional**. O intrare de acest tip poate avea sau nu acest atribut, fără a fi invalidă.

Prin urmare, telephoneNumber este un atribut opțional pentru o intrare de tip inetOrgPerson.

Întrebarea 55

Question: Which of the following is not a valid JDBC url:

Variante de răspuns:

- A. jdbc:mysql://nowhere.net:3306/acme
- B. jdbc://nowhere.net:3306/acme
- C. jdbc:odbc:x

Răspuns corect

B. jdbc://nowhere.net:3306/acme

Explicație

Sintaxa standard și obligatorie pentru un URL JDBC este:

jdbc:<subprotocol>:<subname>

Unde:

- **jdbc:** este un prefix constant care trebuie să apară la începutul fiecărui URL JDBC.
- **<subprotocol>** este numele driver-ului sau al mecanismului de conectare (de exemplu, mysql, oracle, postgresql, odbc). Acesta indică DriverManager-ului ce driver să folosească.
- **<subname>** conține adresa serverului, portul, numele bazei de date și alte proprietăți, iar formatul său depinde de subprotocol.

Să analizăm opțiunile:

- **A. jdbc:mysql://nowhere.net:3306/acme:** Acesta este un URL **valid**. Respectă formatul: jdbc: (prefix), mysql (subprotocol), //nowhere.net:3306/acme (subname).
- **C. jdbc:odbc:x:** Acesta este un URL **valid** (deși corespunde tehnologiei învechite JDBC-ODBC Bridge, eliminată din Java 8). Respectă formatul: jdbc: (prefix), odbc (subprotocol), x (subname, reprezentând un DSN - Data Source Name).
- **B. jdbc://nowhere.net:3306/acme:** Acesta este un URL **invalid**. Îi lipsește partea esențială <subprotocol> între jdbc: și restul adresei (//...). DriverManager nu ar ști ce tip de bază de date este și ce driver să încarce. Formatul corect necesită un subprotocol, ca în exemplul A (jdbc:mysql:...).

Întrebarea 57

Question: The JDBC driver for MySql can be loaded through

Variante de răspuns:

- A. the command line
- B. loading the class in the class loader
- C. registering a Driver instance with the DriverManager
- D. all of the above

Răspuns corect

D. all of the above

Explicație

Există mai multe mecanisme valide prin care un driver JDBC, inclusiv cel pentru MySQL, poate fi încărcat și înregistrat pentru a putea fi folosit de o aplicație. Toate opțiunile enumerate sunt metode valide:

1. **A. the command line (prin linia de comandă):** Se poate utiliza proprietatea de sistem `jdbc.drivers` atunci când se pornește aplicația Java. `DriverManager` va citi această proprietate și va încărca automat clasele specificate. Exemplu: `java -Djdbc.drivers=com.mysql.cj.jdbc.Driver com.myapp.Main`
2. **B. loading the class in the class loader (prin încărcarea clasei în class loader):** Acesta este mecanismul fundamental.
 - **Metoda clasică (pre-JDBC 4.0):** Apelarea explicită `Class.forName("com.mysql.cj.jdbc.Driver");`. Acest lucru încarcă clasa, iar blocul static din interiorul clasei driver-ului se va auto-înregistra la `DriverManager`.
 - **Metoda modernă (JDBC 4.0+):** Driver-ul este descoperit și încărcat automat dacă fișierul său `.jar` se află în `classpath`. Acest lucru funcționează datorită mecanismului `Service Provider Interface (SPI)`.
3. **C. registering a Driver instance with the DriverManager (prin înregistrarea unei instanțe Driver la DriverManager):** Se poate crea manual o instanță a driver-ului și o puteți înregistra direct, folosind cod precum:
`DriverManager.registerDriver(new com.mysql.cj.jdbc.Driver());`

Deoarece toate cele trei metode (A, B și C) sunt căi valide pentru a face driver-ul disponibil aplicației, răspunsul corect este **D. all of the above**.

Întrebarea 58

Question: Given the following code sequence, what will happen if the file cannot be opened:

Java

```
try {  
    conn = DriverManager.getConnection(url, user, password);  
    BufferedReader reader = new BufferedReader(new FileReader("/path/to/file.csv"));  
    //read from file and write to database  
} catch (Exception ex) {  
    Logger.getLogger(Main.class.getName()).log(Level.SEVERE, null, ex);  
} finally{  
    if (conn!=null) conn.close();  
}
```

Variante de răspuns:

- A. an exception will be thrown, but the connection to the database will be created and remain open
- B. an exception will be thrown, but the connection to the database will be created and be closed
- C. the connection will never be created

Răspuns corect

B. an exception will be thrown, but the connection to the database will be created and be closed

Explicație

Să analizăm fluxul de execuție al codului, luând în considerare proprietățile blocului try-catch-finally.

1. Execuția blocului try:

- Prima linie, `conn = DriverManager.getConnection(...)`, se execută cu succes (presupunând că datele de conectare sunt valide). În acest moment, o conexiune la baza de date este creată și este activă, iar variabila `conn` nu mai este null.

- A doua linie, `new FileReader("/path/to/file.csv")`, eșuează deoarece fișierul nu poate fi deschis. Acest eșec aruncă o excepție (de ex., `FileNotFoundException`).
- Când excepția este aruncată, restul codului din blocul `try` este omis, iar controlul trece la blocul `catch`.

2. Execuția blocului `catch`:

- Excepția este prinsă, iar eroarea este înregistrată în log.

3. Execuția blocului `finally`:

- O caracteristică esențială a blocului `finally` este că acesta **se execută întotdeauna** după ce blocurile `try` și `catch` s-au încheiat, indiferent dacă a fost aruncată o excepție sau nu.
- În interiorul blocului `finally`, se verifică `if (conn!=null)`. Deoarece conexiunea a fost creată la primul pas, condiția este adevărată.
- Prin urmare, se execută `conn.close()`, iar conexiunea la baza de date este închisă.

Concluzie:

În secvența de evenimente, se aruncă o excepție, conexiunea este creată înainte de excepție, iar apoi este închisă în mod garantat de blocul `finally`. Acest lucru corespunde exact descrierii din opțiunea B.

Întrebarea 60

Question: Given the following code sequence:

Java

```
PreparedStatement stmt = null;
```

```
stmt = conn.prepareStatement("UPDATE employees SET type = ? WHERE idemployees = ?");
```

```
stmt.setInt(2, 3);
```

```
stmt.executeUpdate();
```

What will happen?

Variante de răspuns:

- A. an exception will be thrown
- B. a warning will be thrown

- C. the "employees" table will be updated

Răspuns corect

A. an exception will be thrown

Explicație

Să analizăm codul pas cu pas:

1. **conn.prepareStatement("UPDATE employees SET type = ? WHERE idemployees = ?");**: Această linie creează un obiect PreparedStatement pentru o comandă SQL care are **doi** parametri, marcați prin semne de întrebare (?). Primul ? este la indexul 1, iar al doilea la indexul 2.
2. **stmt.setInt(2, 3);**: Această linie setează o valoare (numărul întreg 3) pentru cel de-al **doilea** parametru (cel pentru idemployees).
3. **Parametrul lipsă**: Codul **nu** setează nicio valoare pentru **primul** parametru (cel pentru coloana type).
4. **stmt.executeUpdate();**: Se încearcă executarea comenzii.

Conform specificației JDBC, înainte de a executa un PreparedStatement, **toți** parametrii săi trebuie să primească o valoare. Deoarece primul parametru (la indexul 1) nu a fost setat, driver-ul JDBC nu poate trimite o comandă SQL completă către baza de date.

În această situație, la apelul executeUpdate(), driver-ul va arunca o excepție de tip **SQLException**, de obicei cu un mesaj care indică faptul că un parametru nu a fost setat (de exemplu, "No value specified for parameter 1"). Prin urmare, comanda UPDATE nu se va executa deloc.

Întrebarea 75

Întrebare: Care dintre cele de mai jos nu sunt privilegii SYSDBA

Variante de răspuns:

- Startup
- Shutdown
- Hibernate
- Niciunul

Răspuns corect

Hibernate

Explicație

Privilegiul SYSDBA în Oracle acordă control administrativ total asupra instanței bazei de date. Să analizăm opțiunile:

- **STARTUP:** Aceasta este o comandă fundamentală care poate fi executată doar de un utilizator cu privilegii SYSDBA (sau SYSOPER). Este folosită pentru a porni baza de date.
- **SHUTDOWN:** Similar, aceasta este o comandă esențială pentru oprirea bazei de date și necesită privilegii SYSDBA (sau SYSOPER).
- **Hibernate:** Acesta **nu** este un concept sau o comandă standard în administrarea bazelor de date Oracle. "Hibernarea" se referă de obicei la stări ale sistemului de operare sau la framework-uri de persistență a datelor (cum ar fi Hibernate ORM), dar nu reprezintă un privilegiu sau o operațiune directă la nivel de SYSDBA pentru a gestiona starea instanței bazei de date.

Prin urmare, **Hibernate** este opțiunea care nu reprezintă un privilegiu sau o operațiune specifică rolului SYSDBA.

Întrebarea 76

Întrebare: Cine poate interoga v\$pwfile_users?

Variante de răspuns:

- scott
- hr
- student
- sys

Răspuns corect

sys

Explicație

Vizualizarea (view-ul) **v\$pwfile_users** este o vizualizare dinamică de performanță în Oracle care conține informații de securitate foarte sensibile. Ea listează toți utilizatorii cărora li s-au acordat privilegii administrative puternice, precum SYSDBA sau SYSOPER, și care sunt, prin urmare, incluși în fișierul de parole (password file) al bazei de date. Acest fișier permite autentificarea administrativă de la distanță.

Din cauza naturii sale critice pentru securitate, accesul la această vizualizare este extrem de restricționat:

- Utilizatorii obișnuiți, fără privilegii speciale, precum conturile demonstrative **scott**, **hr** sau un cont generic **student**, nu au permisiunea de a o interoga.

- Accesul este rezervat celor mai înalte niveluri de privilegii. Utilizatorul **SYS** este "proprietarul" bazei de date și al dicționarului de date și, prin urmare, are acces implicit la toate aceste obiecte.

Prin urmare, doar utilizatorul **SYS** (sau orice alt utilizator conectat cu privilegiul AS SYSDBA, care în esență acționează ca SYS pentru acea sesiune) poate interoga v\$pwfile_users.

Întrebarea 77

Întrebare: Ce este v\$pwfile_users?

Variante de răspuns:

- tabela virtuala
- partitie
- tabela
- niciun raspuns nu e correct

Răspuns corect

tabela virtuala

Explicație

În Oracle, toate obiectele al căror nume începe cu **v\$** sunt cunoscute ca **vizualizări dinamice de performanță (dynamic performance views)**.

1. O **vizualizare (view)** este, prin definiție, o **tabelă virtuală**. Ea nu stochează date în mod direct, ci este o interogare SQL salvată care prezintă date din una sau mai multe tabele (sau alte vizualizări) într-un format tabular. Când interoghezi o vizualizare, baza de date execută interogarea stocată și îți returnează rezultatul.
2. Vizualizările **dinamice** (precum v\$pwfile_users) sunt un caz special. Ele nu citesc date din tabelele obișnuite de pe disc, ci direct din **structurile de memorie internă** ale instanței Oracle (din SGA - System Global Area). Acestea oferă o imagine în timp real a stării și performanței bazei de date.

Prin urmare, cea mai corectă descriere pentru v\$pwfile_users dintre opțiunile date este **tabelă virtuală**.

- Nu este o **partiție** (o subdiviziune a unei tabele mari).
- Nu este o **tabelă** standard (deoarece nu stochează fizic datele pe care le afișează).

Întrebarea 78

Întrebare: Care comandă este corectă?

Variante de răspuns:

- `select * from v$pwfiles_users;`
- `select count(*) from v$pwfile_users;`
- `select * from v$$pwfile_users;`
- `select * from v$pwfile_end_users;`

Răspuns corect

`select count(*) from v$pwfile_users;`

Explicație

Această întrebare testează cunoașterea numelui exact al vizualizării dinamice de performanță.

Numele corect și complet al vizualizării care afișează utilizatorii din fișierul de parole este **v\$pwfile_users**.

Să analizăm fiecare comandă:

- **`select * from v$pwfiles_users;`** Incorect. Numele conține o greșeală de scriere - pwfiles (plural) în loc de pwfile (singular). Comanda va eșua cu eroarea "ORA-00942: table or view does not exist".
- **`select count(*) from v$pwfile_users;`** Corect. Această comandă folosește numele corect al vizualizării (v\$pwfile_users) și este o interogare SQL validă sintactic, care ar număra utilizatorii din fișierul de parole (presupunând că este executată de un utilizator cu privilegii, precum SYS).
- **`select * from v$$pwfile_users;`** Incorect. Prefixul pentru vizualizările dinamice este v\$, nu v\$\$\$. Comanda va eșua.
- **`select * from v$pwfile_end_users;`** Incorect. Numele conține un text în plus (_end_) și nu corespunde unui obiect real din dicționarul de date Oracle. Comanda va eșua.

Întrebarea 79

Întrebare: Ce reprezintă opțiunea WITH ADMIN OPTION în comanda GRANT?

Variante de răspuns:

- Privilegiu ce poate fi transmis altui utilizator, de către cel ce a primit respectivul privilegiu

- Privilegiu ce poate fi transmis altui utilizator, de către toți utilizatorii, cu sau fără privilegii
- Nu este o opțiune în comanda grant
- Niciun răspuns nu e corect

Răspuns corect

Privilegiu ce poate fi transmis altui utilizator, de către cel ce a primit respectivul privilegiu

Explicație

Clauza **WITH ADMIN OPTION** este folosită în comanda GRANT (în special în Oracle) atunci când se acordă **privilegii de sistem** (ex: CREATE TABLE, CREATE USER) sau **roluri**.

Funcționalitatea sa este următoarea:

Utilizatorul care primește un privilegiu sau un rol cu această opțiune, nu numai că poate folosi el însuși acel privilegiu/rol, dar primește și dreptul de a acorda mai departe (de a transmite) același privilegiu/rol altor utilizatori.

Exemplu:

Dacă se execută comanda: GRANT CREATE SESSION TO utilizator_A WITH ADMIN OPTION;

- utilizator_A va putea să se conecteze la baza de date (acesta este efectul privilegiului CREATE SESSION).
- În plus, utilizator_A va putea să execute și el comanda: GRANT CREATE SESSION TO utilizator_B;, transmițând astfel privilegiul primit.

Prin urmare, prima opțiune descrie corect și precis funcționalitatea clauzei WITH ADMIN OPTION.

Întrebarea 80

Întrebare: Care comandă este corectă?

Variante de răspuns:

- a. REVOKE DELETE ON orders FROM hr;
- b. REVOKE DROP ANY TABLES FROM hr;
- c. REVOKE DROP ON ANY TABLES FROM hr;
- d. REVOKE NOTHING FROM hr;

Răspuns corect

a. REVOKE DELETE ON orders FROM hr;

Explicație

Comanda REVOKE este folosită pentru a retrage privilegii. Sintaxa sa depinde de tipul de privilegiu: privilegiu de obiect sau privilegiu de sistem.

1. **Privilegii de obiect** (ex: SELECT, INSERT, UPDATE, DELETE pe o tabelă anume):

- Sintaxa este: REVOKE <privilegiu> ON <obiect> FROM <utilizator/rol>;

2. **Privilegii de sistem** (ex: CREATE TABLE, DROP ANY TABLE):

- Sintaxa este: REVOKE <privilegiu> FROM <utilizator/rol>; (fără clauza ON).

Să analizăm opțiunile:

- **a. REVOKE DELETE ON orders FROM hr;** Această comandă este **corectă**. DELETE este un privilegiu de obiect, orders este obiectul, iar sintaxa ... ON ... FROM ... este aplicată corect.
- **b. REVOKE DROP ANY TABLES FROM hr;** Această comandă este **incorectă**. Privilegiul de sistem corect este DROP ANY TABLE (la singular). DROP ANY TABLES nu este un privilegiu standard în Oracle.
- **c. REVOKE DROP ON ANY TABLES FROM hr;** Această comandă este **incorectă**. Se încearcă revocarea unui privilegiu de sistem (DROP ANY TABLE), dar se folosește clauza ON, care este rezervată pentru privilegiile de obiect.
- **d. REVOKE NOTHING FROM hr;** Această comandă este **incorectă**. NOTHING nu este un cuvânt cheie sau un privilegiu valid în sintaxa comenzii REVOKE.

Prin urmare, singura comandă formulată corect este cea de la punctul A.

Întrebarea 81

Întrebare: Care din privilegiile de mai jos nu se mai regăsește din Oracle 9i, în sus: 10g, 11g?

Variante de răspuns:

- create table
- create view
- create session
- create procedure

Răspuns și Explicație

Premisa acestei întrebări este **incorrectă**. Toate cele patru privilegii de sistem enumerate mai jos sunt fundamentale în Oracle și **au continuat să existe** în toate versiunile, inclusiv 9i, 10g, 11g și cele mai recente (12c, 19c, 21c, etc.).

Nu există un răspuns corect dintre opțiunile date, deoarece toate sunt valide. Să analizăm rolul fiecăruia:

- **CREATE SESSION:** Este cel mai de bază privilegiu, absolut esențial pentru ca un utilizator să se poată conecta la baza de date.
- **CREATE TABLE:** Este un privilegiu fundamental care permite unui utilizator să creeze tabele în propria schemă.
- **CREATE VIEW:** Este privilegiul standard care permite unui utilizator să creeze vizualizări (view-uri) în propria schemă.
- **CREATE PROCEDURE:** Este privilegiul care permite unui utilizator să creeze proceduri, funcții și pachete PL/SQL în propria schemă.

Toate aceste privilegii sunt piloni ai funcționalității bazei de date Oracle și nu au fost eliminate. Este posibil ca întrebarea să conțină o greșeală.

Întrebarea 82

Întrebare: Ce tabele virtuale pot fi folosite pentru vizualizarea privilegiilor?

Variante de răspuns:

- all_tab_privs_made și user_tab_privs_made;
- all_tab_privs_granted și user_tab_privs_granted;
- all_tab_privs_made și user_tab_privs_made;
- all_tabs_priv_made și user_tabs_priv_made;

Răspuns corect

all_tab_privs_made și user_tab_privs_made (prima și a treia opțiune sunt identice și corecte)

Explicație

Oracle oferă un set de vizualizări (views) în dicționarul de date pentru a inspecta privilegiile. Acestea urmează o convenție de denumire standard. Pentru a vedea privilegiile acordate **de către** un utilizator altor utilizatori, se folosesc vizualizările care se termină în _MADE.

Să analizăm numele corecte:

- **USER_TAB_PRIVS_MADE:** Afișează privilegiile acordate de utilizatorul curent altor utilizatori, pe obiectele pe care utilizatorul curent le deține.
- **ALL_TAB_PRIVS_MADE:** Afișează privilegiile acordate de utilizatorul curent altor utilizatori, pe toate obiectele la care utilizatorul curent are acces (nu doar cele pe care le deține).

Opțiunile din întrebare:

- **Prima și a treia opțiune:** all_tab_privs_made și user_tab_privs_made sunt **corecte**. Acestea sunt numele exacte ale vizualizărilor din dicționarul de date Oracle.
- **A doua opțiune:** ...grated și ...granted sunt **incorecte**. Termenul standard în numele vizualizării este MADE, nu GRANTED.
- **A patra opțiune:** all_tabs_priv_made și user_tabs_priv_made sunt **incorecte**. Numele corecte folosesc TAB (singular) și PRIVS, nu TABS și PRIV.

Prin urmare, prima (și a treia) opțiune listează numele corecte ale tabelelor virtuale (vizualizărilor) folosite pentru a vedea privilegiile acordate de un utilizator.

Întrebarea 83

Întrebare: Privilegiu Caba nu poate fi atribuit rolului

Variante de răspuns:

- Niciun răspuns nu e corect
- PRIVATE
- EXTENDED
- ADVANCED

Răspuns corect

Niciun răspuns nu e corect

Explicație

Această întrebare pare să se bazeze pe termeni care nu sunt standard în sistemele de management al bazelor de date cunoscute, cum ar fi Oracle.

1. **"Privilegiu Caba":** Acesta **nu este un privilegiu standard** sau un concept recunoscut în Oracle sau în alte sisteme de baze de date majore. Este posibil să fie o greșeală de scriere, un termen specific unei anumite aplicații personalizate sau o neînțelegere. Fără o definiție clară a ceea ce reprezintă acest "privilegiu", este imposibil de determinat căror roluri i se poate sau nu atribui.

2. **Rolurile PRIVATE, EXTENDED, ADVANCED:** Acestea, de asemenea, **nu sunt roluri predefinite** în Oracle. Rolurile standard includ CONNECT, RESOURCE, DBA, etc. Rolurile menționate în opțiuni ar fi, cel mai probabil, roluri create de utilizator (CREATE ROLE ...;) într-un mediu specific.

Deoarece atât "privilegiul" cât și "rolurile" menționate sunt non-standard și, posibil, inexistente într-un context general, nu se poate stabili o regulă validă de atribuire între ele. Prin urmare, cea mai logică concluzie este că **niciun răspuns nu este corect**.

Întrebarea 84

Întrebare: Rolul PUBLIC...

Variante de răspuns:

- Este creat de scriptul sql.bsg care se rulează când o bază de date se creează
- Este creat de scriptul sql.bsq care se rulează când o bază de date se creează
- Este creat de scriptul sql.dsq care se rulează când o bază de date se creează
- Nu există rolul PUBLIC

Răspuns corect

Este creat de scriptul sql.bsq care se rulează când o bază de date se creează

Explicație

Rolul PUBLIC este o componentă fundamentală a oricărei baze de date Oracle, fiind creat automat în timpul procesului de creare a bazei de date.

Acest proces implică rularea unor scripturi esențiale care construiesc dicționarul de date (data dictionary).

- Scriptul **sql.bsq** (uneori numit "bootstrap sequence") este unul dintre cele mai importante și de bază scripturi. El conține comenzile DDL (Data Definition Language) pentru crearea tabelor și vizualizărilor fundamentale ale dicționarului de date.
- Crearea entităților de securitate esențiale, precum utilizatorii SYS, SYSTEM și rolul PUBLIC, face parte din acest proces de "bootstrap" inițiat de sql.bsq și continuat de catalog.sql (care la rândul său apelează alte scripturi).

Opțiunile care menționează sql.bsg sau sql.dsq sunt incorecte, deoarece acestea nu sunt numele scripturilor standard pentru acest proces. Afirmția că rolul PUBLIC nu există este, de asemenea, incorectă, deoarece este un concept cheie în modelul de securitate Oracle.

Întrebarea 85

Întrebare: La nivel fizic, organizarea ORACLE este:

Variante de răspuns:

- Database -> tablespace -> segment -> datablock
- Database -> tablespaces -> segments -> datablock
- Database -> tablespaces -> segment -> datablock
- Database -> tablespaces -> segment -> datablocks

Răspuns corect

Database -> tablespaces -> segment -> datablock

Explicație

Această întrebare descrie ierarhia structurilor de stocare logice și fizice din Oracle. Deși toate opțiunile prezintă o oarecare inconsistență în folosirea pluralului, varianta marcată de dvs. poate fi justificată prin următoarea logică:

1. **Database:** Reprezintă întreaga bază de date, containerul de cel mai înalt nivel.
2. **Tablespaces:** O bază de date este formată din una sau mai multe unități logice de stocare numite *tablespace*-uri. Utilizarea pluralului aici este corectă, deoarece o bază de date conține, de regulă, mai multe *tablespace*-uri (SYSTEM, SYSAUX, USERS, TEMP, etc.).
3. **Segment:** În cadrul unui *tablespace*, datele pentru un anumit obiect (o tabelă, un index etc.) sunt stocate într-un *segment*. Aici se poate interpreta că se urmărește calea pentru un singur obiect, deci se folosește singularul.
4. **Datablock:** Un segment este format din unități mai mici numite *extente*, iar extentele sunt formate din cea mai mică unitate de stocare și I/O din Oracle, *blocul de date* (data block). Aici, singularul se referă la unitatea fundamentală de construcție.

Secvența completă este Database -> Tablespace -> Segment -> Extent -> Data Block.

Întrebarea omite pasul "Extent" pentru simplitate. Astfel, ierarhia Database -> tablespaces -> segment -> datablock descrie cum o bază de date conține mai multe *tablespace*-uri, iar apoi urmărește structura unui segment individual până la unitatea sa de bază, blocul de date.

Întrebarea 86

Întrebare: Tablespaces-urile temporare...

Variante de răspuns:

- nu există tablespaces-uri temporare
- n-au tempfiles și au datafiles
- n-au decât datafiles
- au tempfiles și nu datafiles

Răspuns corect

au tempfiles și nu datafiles

Explicație

Această întrebare se referă la diferența fundamentală dintre tablespace-urile permanente și cele temporare în ceea ce privește stocarea fizică în Oracle.

1. **Tablespace-urile Permanente (Permanent Tablespaces):** Acestea sunt folosite pentru a stoca datele permanente ale utilizatorilor și ale sistemului (tabele, indecși, etc.). Din punct de vedere fizic, ele sunt alcătuite din unul sau mai multe fișiere numite **datafiles** (de obicei cu extensia .dbf).
2. **Tablespace-urile Temporare (Temporary Tablespaces):** Acestea sunt folosite de Oracle pentru a gestiona operațiuni care necesită spațiu pe disc pe durata unei sesiuni, cum ar fi sortări masive (ORDER BY, GROUP BY), crearea de indecși, etc. Datele de aici sunt tranzitorii. Pentru a optimiza aceste operațiuni, tablespace-urile temporare nu folosesc datafiles, ci un tip special de fișier numit **tempfile** (de obicei cu extensia .tmp).

Principala diferență este că operațiunile I/O pe tempfiles au un overhead redus (de exemplu, nu se generează la fel de multă informație de redo logging), deoarece datele nu trebuie să fie recuperabile în caz de avarie, spre deosebire de datele din datafiles.

Prin urmare, afirmația corectă este că tablespace-urile temporare **au tempfiles și nu datafiles**.

Întrebarea 86

Întrebare: Tablespaces-urile temporare...

Variante de răspuns:

- nu există tablespaces-uri temporare
- n-au tempfiles și au datafiles
- n-au decât datafiles
- au tempfiles și nu datafiles

Răspuns corect

au tempfiles și nu datafiles

Explicație

Această întrebare se referă la diferența fundamentală dintre tablespace-urile permanente și cele temporare în ceea ce privește stocarea fizică în Oracle.

1. **Tablespace-urile Permanente (Permanent Tablespaces):** Acestea sunt folosite pentru a stoca datele permanente ale utilizatorilor și ale sistemului (tabele, indecși, etc.). Din punct de vedere fizic, ele sunt alcătuite din unul sau mai multe fișiere numite **datafiles** (de obicei cu extensia .dbf).
2. **Tablespace-urile Temporare (Temporary Tablespaces):** Acestea sunt folosite de Oracle pentru a gestiona operațiuni care necesită spațiu pe disc pe durata unei sesiuni, cum ar fi sortări masive (ORDER BY, GROUP BY), crearea de indecși, etc. Datele de aici sunt tranzitorii. Pentru a optimiza aceste operațiuni, tablespace-urile temporare nu folosesc datafiles, ci un tip special de fișier numit **tempfile** (de obicei cu extensia .tmp).

Principala diferență este că operațiunile I/O pe tempfiles au un overhead redus (de exemplu, nu se generează la fel de multă informație de redo logging), deoarece datele nu trebuie să fie recuperabile în caz de avarie, spre deosebire de datele din datafiles.

Prin urmare, afirmația corectă este că tablespace-urile temporare **au tempfiles și nu datafiles**.

În regulă, acum am întrebarea completă.

Întrebarea 88

Întrebare: Se dă partiția

SQL

```
create table sales (year number(4),  
                    product varchar2(10),  
                    val number(10,2))  
partition by range (year)  
(partition p1 values less than (1992) tablespace u1,  
partition p2 values less than (1993) tablespace u2,  
partition p3 values less than (1994) tablespace u3,  
partition p4 values less than (1995) tablespace u4,  
partition p5 values less than (MAXVALUE) tablespace u5);
```


Care afirmație e corectă?

Variante de răspuns:

- Partitia nu e bine definita
- Partitia nu are definita MINVALUE
- Nu se pot introduce date in tabela partitionata mai mari decat MAXVALUE., unde MAXVALUE e max(coloana)
- Partitia nu are definit MAXVALUE corect

Răspuns corect

Partiția nu are definită MINVALUE

Explicație

Să analizăm fiecare afirmație în contextul codului SQL furnizat:

1. **Partiția nu e bine definită: Fals.** Comanda CREATE TABLE este sintactic și logic corectă. Ea definește o tabelă funcțională, partiționată prin interval (range) pe coloana year, cu partiții clare și non-suprapuse.
2. **Nu se pot introduce date ... mai mari decat MAXVALUE: Fals.** Această afirmație denotă o neînțelegere a cuvântului cheie MAXVALUE. MAXVALUE nu este o valoare calculată (precum max(coloana)), ci un cuvânt special în Oracle care reprezintă o valoare infinită teoretică. Partiția p5 va accepta orice valoare pentru year mai mare sau egală cu 1995, până la cea mai mare valoare posibilă. Prin definiție, nu se poate introduce o valoare "mai mare decât MAXVALUE".
3. **Partiția nu are definit MAXVALUE corect: Fals.** Folosirea VALUES LESS THAN (MAXVALUE) pentru ultima partiție este metoda standard și corectă de a crea o partiție "catch-all" care să preia toate valorile peste ultima limită definită.
4. **Partiția nu are definită MINVALUE: Adevărat.** Aceasta este o afirmație factuală despre sintaxa partiționării prin interval în Oracle. Partițiile sunt definite doar prin limita lor **superioară** (VALUES LESS THAN ...). Nu există un cuvânt cheie MINVALUE pentru a defini limita inferioară a primei partiții; aceasta este considerată implicit "minus infinit". Deși lipsa MINVALUE nu este o eroare (așa funcționează sintaxa), afirmația în sine — că MINVALUE nu este definit explicit în script — este adevărată. Dintre cele patru opțiuni, aceasta este singura afirmație corectă din punct de vedere factual.

Întrebarea 89

Întrebare: Care este relația între cluster și partition, la nivel fizic?

Variante de răspuns:

- nu există nicio relație
- de incluziune
- de excluziune
- niciun răspuns nu e corect

Răspuns corect

nu există nicio relație

Explicație

Aceasta este o întrebare subtilă care se referă la două concepte de organizare a datelor în Oracle, care sunt în mare parte independente (ortogonale).

1. **Partiționarea (Partitioning):** Este o tehnică prin care o tabelă mare este împărțită logic în bucăți mai mici (partiții) pe baza unei chei. Acest lucru se face în principal pentru a îmbunătăți administrarea (manageability) și performanța interogărilor (prin partition pruning). Fiecare partiție este stocată, în general, în propriul său segment fizic.
2. **Clustering-ul (Clustering):** Este o tehnică prin care date din una sau mai multe tabele care sunt frecvent accesate împreună (de obicei printr-un join) sunt stocate fizic în aceleași blocuri de date. Acest lucru se face pentru a reduce operațiunile de I/O și a accelera join-urile.

Relația dintre ele:

- Cele două concepte **nu sunt mutual exclusive**. O tabelă poate fi atât partiționată, cât și parte a unui cluster (clustered).
- Ele adresează probleme diferite și sunt, în esență, **caracteristici independente** pe care un DBA le poate folosi. Nu există o relație ierarhică sau de dependență între ele, așa cum există, de exemplu, între un tablespace și un segment (unde segmentul este conținut în tablespace).
- **Nu este o relație de incluziune**, deoarece un cluster nu "conține" o partiție ca obiect, și nici invers.
- **Nu este o relație de excluziune**, deoarece pot fi folosite împreună.

Deoarece sunt două mecanisme de organizare fizică distincte și independente, care pot fi combinate, cea mai corectă descriere dintre opțiunile date este că **nu există nicio relație** directă (ierarhică) între ele.

Întrebarea 90

Întrebare: EUS vine de la

Variante de răspuns:

- Enterprise Users Security
- Enterprise User Security
- Enterprises Unregistered Security
- Enterprise Unified Security

Răspuns corect

Enterprise User Security

Explicație

EUS (Enterprise User Security) este o componentă a Oracle Database care permite administrarea centralizată a utilizatorilor de baze de date.

În loc de a crea și gestiona conturi de utilizator separat în fiecare bază de date dintr-o organizație, EUS permite definirea utilizatorilor o singură dată într-un serviciu de director, de obicei **Oracle Internet Directory (OID)**. Acești "utilizatori de întreprindere" (enterprise users) pot fi apoi mapați la scheme partajate în diverse baze de date, iar privilegiile și rolurile lor sunt, de asemenea, gestionate centralizat în director.

Beneficiile principale sunt simplificarea administrării, aplicarea consistentă a politicilor de securitate și o reducere a costurilor operaționale în medii cu un număr mare de baze de date.

Întrebarea 91

Întrebare: Securitatea bazei de date este asigurată la nivelul integrității datelor prin intermediul:

Variante de răspuns:

- DDL: grant/priv
- DML: grant/insert/delete
- DML grant/revoke
- DDL: grant/revoke

Răspuns corect

DDL: grant/revoke

Explicație

Această întrebare leagă concepte de securitate, integritate și tipuri de comenzi SQL.

1. **Protejarea integrității datelor:** O modalitate fundamentală de a proteja integritatea datelor (asigurarea că acestea sunt corecte și consistente) este prin a controla cine are dreptul să le modifice. Acest control al accesului este un pilon al securității bazelor de date.
2. **Comenzile de control:** Comenzile SQL folosite pentru a gestiona permisiunile și drepturile de acces sunt **GRANT** (pentru a acorda privilegii) și **REVOKE** (pentru a retrage privilegii).
3. **Clasificarea comenzilor:**
 - **DML (Data Manipulation Language):** Se referă la comenzile care manipulează direct datele din tabele: SELECT, INSERT, UPDATE, DELETE.
 - **DDL (Data Definition Language):** Se referă la comenzile care definesc sau modifică structura obiectelor din baza de date: CREATE, ALTER, DROP.
 - **DCL (Data Control Language):** Aceasta este categoria cea mai precisă pentru GRANT și REVOKE, deoarece aceste comenzi controlează accesul la date.

În multe contexte academice simplificate, comenzile DCL sunt grupate împreună cu cele **DDL**, deoarece GRANT și REVOKE modifică dicționarul de date (metadata despre permisiuni), similar cu modul în care comenzile DDL modifică metadata despre structura tabelor. Ele clar **nu sunt** comenzi DML.

Analizând opțiunile, **DDL: grant/revoke** este cea mai corectă alegere:

- Identifică corect comenzile GRANT și REVOKE ca fiind mecanismul de securitate.
- Le încadrează în categoria DDL (o încadrare acceptabilă și mult mai corectă decât DML).

Întrebarea 92

Întrebare: DES sau:

Variante de răspuns:

- Database encryption standard
- Data encryption support
- Data encryption standard
- Data encoded standard

Răspuns corect

Data Encryption Standard

Explicație

DES este acronimul pentru **Data Encryption Standard** (Standardul de Criptare a Datelor).

A fost un algoritm de criptare simetrică extrem de influent, selectat ca standard federal de către guvernul Statelor Unite în 1977. Timp de mulți ani, a fost cel mai utilizat algoritm de criptare la nivel mondial.

Astăzi, din cauza lungimii sale reduse a cheii (56 de biți), DES este considerat **nesigur** în fața atacurilor moderne și a fost înlocuit în majoritatea aplicațiilor de algoritmi mai robuști, cel mai notabil fiind **AES (Advanced Encryption Standard)**.

Întrebarea 93

Întrebare: Comanda `idabindentry` este folosită...

Variante de răspuns:

- a. Pentru autentificare anonimă
- b. Pentru autentificare cu un `entry(user)` folosit ca atribut
- c. Nu este o comandă LDAP
- d. Poate fi folosită pentru mai multe autentificări anonime

Răspuns corect

c. Nu este o comandă LDAP

Explicație

Comanda **`idabindentry`** nu este o comandă standard sau recunoscută în seturile de unelte pentru lucrul cu directoare LDAP (cum ar fi cele din OpenLDAP, Oracle Internet Directory sau Active Directory).

Comenzile standard de linie de comandă pentru a interacționa cu un server LDAP includ:

- `ldapsearch` (pentru căutare)
- `ldapadd` (pentru adăugare de intrări)
- `ldapmodify` (pentru modificare)
- `ldapdelete` (pentru ștergere)
- **`ldapbind`** (pentru autentificare)

Operațiunea de autentificare în LDAP se numește **BIND**. Comanda de linie de comandă corespunzătoare este **ldapbind**. Aceasta poate fi folosită atât pentru autentificare anonimă (de obicei cu opțiunea -x), cât și pentru autentificare cu un utilizator specific (specificând DN-ul și parola).

Deoarece idabindentry nu este o comandă reală în contextul uneltelor LDAP standard, răspunsul corect este că **nu este o comandă LDAP**.

Întrebarea 94

Întrebare: Autentificare cu un user specific se face folosind următoarea comandă:

Variante de răspuns:

- a. Ldapbind -h host -p port -D "" -W userpassword
- b. Ldapbind -h host -p port -D "userdn" -w userpassword
- c. Ldapbind -h host -p port -G "" -W userpassword
- d. Ldapbind -h host -p port -G "userdn" -W userpassword

Răspuns corect

b. Ldapbind -h host -p port -D "userdn" -w userpassword

Explicație

Comanda standard pentru autentificare (bind) în LDAP este **ldapbind**. Pentru a te autentifica drept un utilizator specific, trebuie să furnizezi credențialele acestuia, care constau de obicei într-un DN (Distinguished Name) și o parolă.

Să analizăm opțiunile standard ale comenzii ldapbind:

- **-h <host>**: specifică serverul LDAP.
- **-p <port>**: specifică portul serverului.
- **-D <bindDN>**: specifică **Distinguished Name-ul** utilizatorului cu care se face autentificarea. Acesta este parametrul cheie pentru a indica un utilizator specific.
- **-w <parola>**: specifică parola pentru bindDN, direct în linia de comandă.
- **-W**: solicită interactiv introducerea parolei (mai sigur decât -w).

Analizând variantele:

- **a.** Folosește -D "" (DN gol), ceea ce corespunde unei autentificări **anonime**, nu cu un utilizator specific.

- **b.** Folosește corect **-D "userdn"** pentru a specifica utilizatorul și **-w userpassword** pentru a-i furniza parola. Aceasta este sintaxa corectă pentru o autentificare simplă cu un utilizator specific.
- **c. și d.** Folosesc opțiunea **-G**, care nu este un parametru standard pentru specificarea utilizatorului în **ldapbind**. Parametrul corect este **-D**.

Prin urmare, comanda de la punctul B este singura formulată corect pentru autentificarea cu un utilizator specific.

Întrebarea 95

Întrebare: În JDBC, comenzile SQL simple se utilizează prin obiecte:

Variante de răspuns:

- a. Statement
- b. PreparedStatement
- c. CallableStatement
- d. ResultSet

Răspuns corect

a. Statement

Explicație

API-ul JDBC oferă diferite tipuri de obiecte pentru a executa comenzi SQL, fiecare cu un scop specific:

- **Statement:** Acesta este obiectul de bază, folosit pentru a executa **comenzi SQL simple, statice**, care nu conțin parametri. Este ideal pentru interogări care se execută o singură dată sau rar. Exemplu: `statement.executeQuery("SELECT * FROM angajati");`.
- **PreparedStatement:** Este folosit pentru a executa comenzi SQL parametrizate (care conțin ? ca înlocuitori) și pre-compilate. Este mai performant pentru comenzi executate repetat și esențial pentru securitate (previne atacurile de tip SQL Injection).
- **CallableStatement:** Este specializat pentru a apela proceduri stocate (stored procedures) din baza de date.
- **ResultSet:** Nu este un obiect pentru a executa comenzi. Este obiectul care **conține rezultatele** returnate de o comandă SELECT.

Prin urmare, pentru "comenzi SQL simple" (statice, fără parametri), obiectul corect de utilizat este **Statement**.

Întrebarea 96

Întrebare: Pentru evitarea atacurilor prin acces direct la serverul de baze de date nu este utilă:

Variante de răspuns:

- a. Criptarea conexiunii
- b. Criptarea la nivelul de linie
- c. Criptarea la nivel de tabelă
- d. Criptarea la nivel de coloană

Răspuns corect

a. Criptarea conexiunii

Explicație

Întrebarea se referă la protecția împotriva unui atacator care a obținut **acces direct la server**, ceea ce implică de obicei acces la sistemul de fișiere unde sunt stocate fizic datele bazei de date (fișierele .dbf, .ibd, etc.). Aceasta este o amenințare la adresa datelor în repaus ("data-at-rest").

Să analizăm utilitatea fiecărei măsuri în acest context:

- **b, c, d (Criptarea la nivel de linie, tabelă, coloană):** Toate acestea sunt forme de **criptare a datelor în repaus (data-at-rest encryption)**. Scopul lor este exact acesta: să protejeze datele stocate pe disc. Chiar dacă un atacator fură fișierele fizice ale bazei de date, el nu va putea citi datele deoarece acestea sunt criptate. Prin urmare, aceste măsuri sunt **foarte utile**.
- **a. Criptarea conexiunii:** Aceasta este o măsură de protecție pentru **datele în tranzit (data-in-transit)**, de obicei implementată prin SSL/TLS. Scopul ei este de a proteja datele în timp ce călătoresc prin rețea între client și server, prevenind atacuri de tip "man-in-the-middle" sau "eavesdropping" (interceptarea traficului).

Concluzie:

Dacă un atacator are deja acces direct la server, el a depășit deja perimetrul protejat de rețea. Criptarea conexiunii nu mai oferă nicio protecție pentru fișierele care se află deja pe discul serverului. Prin urmare, în acest scenariu specific, criptarea conexiunii nu este utilă.

Întrebarea 99

Întrebare: Care dintre afirmații este adevărată?

Variante de răspuns:

- a. În FN1 pot exista câmpuri redundante
- b. În FN2 nu există dependențe tranzitive
- c. În FN3 pot exista dependențe tranzitive
- d. În FN3 nu există dependențe parțiale

Răspuns și Explicație

Această întrebare este problematică, deoarece **două dintre afirmații (a și d) sunt tehnic adevărate**. Totuși, într-un context de test, de obicei se caută cea mai riguroasă sau definitorie afirmație.

Să analizăm fiecare opțiune:

- **a. În FN1 pot exista câmpuri redundante**
 - **Adevărat.** Prima Formă Normală (FN1) asigură doar că valorile sunt atomice (nu există grupuri repetitive). Ea nu rezolvă problemele de redundanță a datelor cauzate de dependențe funcționale, acestea fiind adresate de FN2 și FN3.
- **b. În FN2 nu există dependențe tranzitive**
 - **Fals.** O tabelă în FN2 a eliminat dependențele parțiale, dar poate încă să conțină dependențe tranzitive. Eliminarea acestora este scopul FN3.
- **c. În FN3 pot exista dependențe tranzitive**
 - **Fals.** Prin însăși definiția sa, o tabelă în FN3 nu are voie să aibă dependențe tranzitive.
- **d. În FN3 nu există dependențe parțiale**
 - **Adevărat.** Formele normale sunt ierarhice și cumulative. Pentru ca o tabelă să ajungă în FN3, trebuie mai întâi să fie în FN2. O condiție esențială pentru FN2 este eliminarea tuturor dependențelor parțiale. Prin urmare, este garantat că o tabelă în FN3 nu are dependențe parțiale.

Concluzie:

Atât afirmația **a** cât și afirmația **d** sunt corecte. Totuși, dacă trebuie aleasă o singură variantă, **afirmația (d) este considerată adesea mai riguroasă** deoarece descrie o proprietate strictă, garantată de ierarhia definițiilor formale ale normalizării (FN3 => FN2

=> Fără dependențe parțiale). Afirmatia (a) este de asemenea corectă, dar descrie mai degrabă o problemă pe care FN1 nu o rezolvă.

Întrebarea 100

Care afirmație este adevărată în modelul relațional?

- a. Atributul este o tabelă
- b. Atributul este o linie a unei tabele
- c. Atributul este o coloană a unei relații
- d. Atributul este un tip de date

Răspuns: c. Atributul este o coloană a unei relații

Explicație:

În modelul relațional, conceptele cheie sunt definite astfel:

- **Relație (Tabelă):** Reprezintă o colecție de date organizate în rânduri și coloane.
- **Atribut (Coloană):** O caracteristică sau proprietate specifică a entității pe care o descrie relația. Fiecare atribut are un nume unic în cadrul relației și un domeniu (tip de date) din care își ia valorile. De exemplu, într-o tabelă "Studenti", "Nume", "Prenume" și "Vârstă" ar fi atribute.
- **Tuplu (Linie/Înregistrare):** Un rând într-o relație, care reprezintă o singură instanță a entității și conține valorile specifice pentru fiecare atribut.
- **Domeniu (Tip de date):** Setul de valori permise pentru un atribut.

Prin urmare, afirmația că un atribut este o coloană a unei relații este cea corectă, deoarece fiecare coloană dintr-o tabelă relațională reprezintă un atribut al entității modelate.

Întrebarea 101

Este o comandă DML?

- a. INSERT
- b. CREATE
- c. ALTER TABLE
- d. DROP TABLE

Răspuns: a. INSERT

Explicație:

Limbajul SQL (Structured Query Language) este împărțit în mai multe subcategorii, printre care:

- **DML (Data Manipulation Language - Limbajul de Manipulare a Datelor):**

Aceste comenzi sunt utilizate pentru a manipula datele stocate în baza de date. Ele afectează rândurile individuale dintr-o tabelă.

- **INSERT:** Adaugă rânduri noi într-o tabelă.
- **UPDATE:** Modifică datele existente în rândurile unei tabele.
- **DELETE:** Șterge rânduri dintr-o tabelă.
- **SELECT:** Regăsește date dintr-o bază de date (deși unii îl clasifică separat ca DQL - Data Query Language, este adesea inclus în DML).

- **DDL (Data Definition Language - Limbajul de Definire a Datelor):**

Aceste comenzi sunt utilizate pentru a defini sau a modifica structura obiectelor bazei de date (tabele, view-uri, indecși etc.).

- **CREATE:** Creează noi obiecte în baza de date (ex: CREATE TABLE, CREATE DATABASE).
- **ALTER:** Modifică structura unui obiect existent (ex: ALTER TABLE).
- **DROP:** Șterge obiecte din baza de date (ex: DROP TABLE, DROP DATABASE).
- **TRUNCATE:** Șterge toate rândurile dintr-o tabelă, dar păstrează structura tablei.

Prin urmare, dintre opțiunile date, INSERT este singura comandă DML.

Întrebarea 102

Este posibil un atac prin SQL Injection dacă se utilizează?

- a. Obiect Statement
- b. Obiecte PreparedStatement
- c. Proceduri stocate
- d. Obiecte CallableStatement

Răspuns: a. Obiect Statement

Explicație:

Atacurile de tip **SQL Injection** apar atunci când datele introduse de utilizator nu sunt validate sau "escaped" corect, permițând unui atacator să insereze fragmente de cod

SQL malicios în interogările bazei de date. Scopul este de a manipula interogarea originală pentru a obține acces neautorizat, a modifica date sau a efectua alte operațiuni nedorite.

Iată de ce varianta **a. Object Statement** este vulnerabilă și de ce celelalte opțiuni sunt metode de prevenție:

- **a. Object Statement:** Când folosești un obiect Statement în Java (sau echivalente în alte limbaje, cum ar fi concatenarea directă a șirurilor de caractere), valorile variabilelor sunt concatenate direct în șirul de interogare SQL. Acest lucru face aplicația extrem de vulnerabilă la SQL Injection, deoarece orice caracter special introdus de utilizator (cum ar fi ghilimele simple, punct și virgulă, comentarii SQL -- sau /* */) poate schimba logica interogării SQL.
- **b. Obiecte PreparedStatement:** Acestea sunt proiectate special pentru a preveni atacurile SQL Injection. PreparedStatement compilează interogarea SQL o singură dată, iar ulterior, valorile variabilelor sunt trimise separat la baza de date și "legate" în interogare. Baza de date tratează aceste valori ca date pure, nu ca parte a codului SQL. Astfel, chiar dacă un atacator introduce caractere speciale, acestea vor fi interpretate ca parte a datelor, nu ca instrucțiuni SQL.
- **c. Proceduri stocate (Stored Procedures):** Atunci când sunt implementate corect și utilizează parametrizare (similar cu PreparedStatement), procedurile stocate pot oferi o protecție excelentă împotriva SQL Injection. Dacă procedura stocată utilizează parametri expliți pentru intrări (în loc să concateneze șiruri), riscul de injecție este redus semnificativ. Totuși, o procedură stocată care concatenează șiruri de intrare (utilizând, de exemplu, EXEC sau sp_executesql fără parametrizare adecvată) poate fi la fel de vulnerabilă. Cheia este utilizarea corectă a parametrilor.
- **d. Obiecte CallableStatement:** Sunt folosite pentru a executa proceduri stocate. La fel ca PreparedStatement, CallableStatement permite trimiterea parametrilor în siguranță, prevenind SQL Injection, atâta timp cât procedura stocată subiacentă este și ea securizată prin utilizarea corectă a parametrilor.

În concluzie, principala cauză a vulnerabilităților SQL Injection este concatenarea nesigură a intrărilor de utilizator în interogările SQL. PreparedStatement și utilizarea corectă a parametrilor în procedurile stocate (accesate prin CallableStatement) sunt cele mai eficiente metode de prevenție.

Întrebarea 103

Cheile primare

a. Pot avea valori nedefinite

- b. Identifică unic un tuplu
- c. Identifică unic o coloană
- d. Pot avea valori duplicate

Răspuns: b. Identifică unic un tuplu

Explicație:

În modelul relațional al bazelor de date, o **cheie primară (Primary Key)** este un concept fundamental cu roluri și proprietăți bine definite:

- **Identificare Unică a Tuplurilor (Rândurilor):** Rolul principal al unei chei primare este de a identifica în mod unic fiecare rând (tuplu) dintr-o tabelă. Aceasta înseamnă că nu pot exista două rânduri diferite în aceeași tabelă care să aibă aceeași valoare pentru cheia primară.
- **Unicitate:** Valorile dintr-o cheie primară trebuie să fie unice în cadrul tabelii.
- **Non-Null (Nu pot fi nedefinite):** O cheie primară nu poate conține valori NULL (nedefinite). Fiecare rând trebuie să aibă o valoare validă și cunoscută pentru cheia primară. Această proprietate este cunoscută sub numele de "integritate a entității".
- **Stabilitate:** Ideal, valorile cheii primare nu ar trebui să se schimbe.

Să analizăm celelalte opțiuni:

- **a. Pot avea valori nedefinite:** Această afirmație este falsă. Cheile primare nu pot avea valori NULL.
- **c. Identifică unic o coloană:** Această afirmație este falsă. Cheile primare identifică rânduri (tupluri), nu coloane. Numele coloanelor sunt unice în cadrul unei tabele, dar nu sunt identificate de o cheie primară.
- **d. Pot avea valori duplicate:** Această afirmație este falsă. Prin definiție, valorile cheii primare trebuie să fie unice.

Prin urmare, afirmația corectă este că o cheie primară identifică unic un tuplu (rând) într-o tabelă.

Întrebarea 104

Nu este o comandă MySQL de management al rolurilor (sau utilizatorilor/permisiunilor)?

- a. CREATE USER
- b. REVOKE
- c. GRANT

d. DELETE USER

Răspuns: d. DELETE USER

Explicație:

În MySQL, managementul utilizatorilor, rolurilor și permisiunilor se realizează prin comenzi specifice, care fac parte din DCL (Data Control Language - Limbajul de Control al Datelor).

Să analizăm opțiunile:

- **a. CREATE USER:** Această comandă este folosită pentru a crea noi conturi de utilizator în MySQL. Este o comandă fundamentală pentru managementul securității și al utilizatorilor.
- **b. REVOKE:** Această comandă este utilizată pentru a revoca (retrage) permisiuni (privilegii) acordate anterior unui utilizator sau unui rol. Este esențială pentru managementul accesului și al securității.
- **c. GRANT:** Această comandă este utilizată pentru a acorda permisiuni (privilegii) specifice unui utilizator sau unui rol. Este, de asemenea, o comandă cheie în managementul securității.
- **d. DELETE USER:** Această comandă *nu există* în sintaxa standard MySQL pentru ștergerea unui utilizator. Comanda corectă pentru a șterge un utilizator în MySQL este **DROP USER**.

Deși DELETE este o comandă DML (Data Manipulation Language) utilizată pentru a șterge rânduri dintr-o tabelă, ea nu este folosită pentru a șterge conturi de utilizator din baza de date însăși. Sintaxa corectă pentru managementul utilizatorilor implică CREATE USER, DROP USER, GRANT, REVOKE, etc.

Prin urmare, DELETE USER nu este o comandă validă de management al utilizatorilor/rolurilor în MySQL.

Întrebarea 105

O tabelă în 1NF (Prima Formă Normală) nu conține:

- Transitive dependencies
- Null values
- Repeating groups

Răspuns: Repeating groups

Explicație:

Întrebarea se referă la **Prima Formă Normală (1NF)**, care este fundamentul normalizării bazelor de date relaționale. Deși ați scris "FN1", este o presupunere rezonabilă că ați vrut să spuneți "1NF".

Pentru ca o tabelă să fie în Prima Formă Normală (1NF), trebuie să îndeplinească următoarele condiții:

1. **Atomic Assets (Valori Atomice):** Fiecare celulă a tabelului (intersecția dintre un rând și o coloană) trebuie să conțină o singură valoare, indivizibilă (atomică). Aceasta înseamnă că nu ar trebui să existe liste de valori sau valori multiple într-o singură celulă.
2. **No Repeating Groups (Fără Grupuri Repetate):** Nu trebuie să existe grupuri de coloane repetate. Dacă o tabelă conține coloane care se repetă (ex: Telefon1, Telefon2, Telefon3), sau seturi de coloane care ar trebui să fie într-o tabelă separată, atunci nu este în 1NF. Fiecare rând trebuie să aibă același număr de coloane.
3. **Unique Primary Key (Cheie Primară Unică):** Fiecare rând trebuie să poată fi identificat în mod unic printr-o cheie primară.

Să analizăm opțiunile date în contextul 1NF:

- **Transitive dependencies (Dependențe tranzitive):** O dependență tranzitivă există atunci când o coloană non-cheie depinde de o altă coloană non-cheie, care, la rândul său, depinde de cheia primară ($A \rightarrow B$ și $B \rightarrow C$, deci $A \rightarrow C$). Prezența dependențelor tranzitive împiedică o tabelă să fie în **A Treia Formă Normală (3NF)**, dar *nu* în 1NF sau 2NF. O tabelă în 1NF poate avea încă dependențe tranzitive.
- **Null values (Valori nule):** O tabelă în 1NF poate conține valori nule (NULL). Regula 1NF se referă la structura datelor (atomicitate și grupuri repetate), nu la prezența sau absența valorilor lipsă. Deși este o bună practică de design să minimizezi valorile NULL, ele sunt permise în 1NF.
- **Repeating groups (Grupuri repetate):** Aceasta este o violare directă a Primei Forme Normale. Dacă o tabelă conține grupuri repetate de atribute, atunci ea *nu* este în 1NF. Eliminarea grupurilor repetate este unul dintre pașii cheie pentru a aduce o tabelă în 1NF.

Prin urmare, o tabelă care respectă Prima Formă Normală **nu conține grupuri repetate**.

Întrebarea 106

Un obiect CallableStatement este utilizat pentru:

- The execution of a stored procedure (Execuția unei proceduri stocate)

- The execution of a DB function (Execuția unei funcții de bază de date)
- The execution of a DB trigger (Execuția unui trigger de bază de date)

Răspuns: The execution of a stored procedure (Execuția unei proceduri stocate)

Explicație:

În Java JDBC (Java Database Connectivity), obiectul **CallableStatement** este o interfață specifică, derivată din **PreparedStatement**, care este concepută pentru a executa **proceduri stocate** și **funcții stocate** (DB functions) în baza de date. Acesta permite apelarea procedurilor și funcțiilor care pot avea parametri de intrare, de ieșire sau atât de intrare, cât și de ieșire.

Să detaliem opțiunile:

- **The execution of a stored procedure (Execuția unei proceduri stocate):**
Aceasta este utilizarea principală a unui **CallableStatement**. Procedurile stocate sunt blocuri de cod SQL precompilate și stocate în baza de date. **CallableStatement** oferă o modalitate sigură și eficientă de a le executa dintr-o aplicație Java, gestionând parametrii și valorile returnate.
- **The execution of a DB function (Execuția unei funcții de bază de date):** Da, **CallableStatement** poate fi folosit și pentru a executa funcții stocate (care sunt un tip de procedură stocată, dar returnează o singură valoare). De exemplu, poți înregistra un parametru de ieșire pentru valoarea returnată de funcție.
- **The execution of a DB trigger (Execuția unui trigger de bază de date):** Nu, **CallableStatement** *nu* este utilizat pentru a executa triggere (declanșatoare) de bază de date. **Triggerele** sunt blocuri de cod SQL care se execută **automat** ca răspuns la anumite evenimente (cum ar fi INSERT, UPDATE, DELETE) pe o tabelă. Ele sunt gestionate intern de sistemul de management al bazei de date (DBMS) și nu sunt apelate direct dintr-o aplicație prin JDBC sau alte API-uri.

Prin urmare, deși **CallableStatement** poate executa atât proceduri, cât și funcții stocate, contextul întrebării implică o alegere din cele oferite. Execuția unui **trigger** este fundamentalmente diferită și nu se face prin **CallableStatement**.

Întrebarea 107

DIT înseamnă:

- Directory Information Terminal
- Direction Intersection Terminology
- Directory Information Technique
- Directory Information Tree

Răspuns: d. Directory Information Tree

Explicație:

În contextul bazelor de date și al sistemelor de directoare (în special cele bazate pe standardul LDAP - Lightweight Directory Access Protocol), **DIT** este un acronim pentru **Directory Information Tree**.

Un **Directory Information Tree (DIT)** reprezintă structura ierarhică în care sunt organizate intrările dintr-un director LDAP. Gândește-te la el ca la o structură de arbore, similară cu structura de foldere și fișiere de pe un sistem de operare, unde:

- **Rădăcina (Root):** Este punctul de plecare al arborelui.
- **Intrări (Entries):** Fiecare nod din arbore este o intrare, care reprezintă un obiect (de exemplu, o persoană, un grup, o organizație, o imprimantă etc.). Fiecare intrare are attribute asociate (cum ar fi nume, adresă de e-mail, număr de telefon).
- **Relații Părinte-Copil:** Intrările sunt organizate într-o structură ierarhică, unde o intrare poate fi părintele altor intrări (copii).

Această structură arborescentă permite organizarea și regăsirea eficientă a informațiilor în directoare. De exemplu, în LDAP, un "Distinguished Name" (DN) identifică în mod unic o intrare în DIT prin specificarea căii complete de la rădăcină la acea intrare.

Celelalte opțiuni nu reprezintă acronime standard recunoscute în domeniul IT sau al bazelor de date.

Întrebarea 108

2. În structura de mai sus: care este root-ul (rădăcina) OID-ului de mai sus?

- a. cn=users
- b. dc=com,dc=erednic
- c. dc=erednic,dc=com
- d. cn=users,dc=erednic,dc=com

Răspuns: c. dc=erednic,dc=com

Explicație:

Structura prezentată este un arbore de informații de director (Directory Information Tree - DIT) specific Oracle Internet Directory (OID), care este o implementare a unui server LDAP. În LDAP, intrările sunt organizate ierarhic, iar "rădăcina" sau "baza" directorului (Base DN) este punctul de plecare al acestei ierarhii pentru o anumită organizație sau domeniu.

Să analizăm imaginea și opțiunile:

- **Structura vizibilă în imagine:** Sub secțiunea "Entry Management", observăm o ierarhie care începe cu dc=com, iar sub dc=com se află dc=erednic. Această structură dc=erednic,dc=com reprezintă un nume de domeniu distinct (Distinguished Name - DN) complet, unde erednic este un subdomeniu al com. Acesta este modul standard în care sunt reprezentate domeniile în LDAP, corespunzând de obicei numelui de domeniu al organizației (ex: erednic.com).
- **Analiza opțiunilor:**
 - **a. cn=users:** Acesta este un container (o ramură) pentru utilizatori și se află la un nivel mult mai jos în ierarhie (cn=users este sub dc=erednic). Nu este rădăcina.
 - **b. dc=com,dc=erednic:** Aceasta este o ordine incorectă a componentelor unui DN. În LDAP, componentele DN sunt listate de la cele mai specifice la cele mai generale, separate prin virgule. Ordinea corectă ar fi dc=erednic,dc=com.
 - **c. dc=erednic,dc=com:** Aceasta reprezintă numele distinct al bazei directorului pentru domeniul erednic.com. Din structura arborescentă, se vede că toate celelalte intrări relevante pentru organizație (cum ar fi cn=Users, cn=Groups, etc.) sunt sub această ramură. Prin urmare, aceasta este rădăcina logică sau baza OID-ului pentru conținutul specific al organizației "erednic".
 - **d. cn=users,dc=erednic,dc=com:** Acesta este un DN complet pentru containerul de utilizatori (cn=users) din domeniul erednic.com. Este o intrare specifică în director, nu rădăcina acestuia.

Prin urmare, dc=erednic,dc=com este punctul cel mai înalt al arborelui de informații de director care reprezintă structura organizațională specifică "erednic", făcându-l "rădăcina" în contextul dat.

Întrebarea 109

3. Câți useri sunt în containerul cn=users,dc=erednic,dc=com?

- a. 3
- b. 2
- c. 1, doar cel selectat
- d. nici unul

Răspuns: b. 2

Explicație:

În structura de director (Directory Information Tree - DIT) prezentată, containerul care ne interesează este cn=Users (presupunând o mică eroare de scriere în întrebare, "userr" în loc de "users"). Acest container este situat sub dc=erednic,dc=com.

Privind ramura cn=Users din imagine, observăm următoarele intrări directe sub el:

1. cn=erednic (aceasta este intrarea selectată în imagine, indicând un utilizator sau administrator specific)
2. cn=orcladmin (acesta este utilizatorul superadministrator al Oracle Internet Directory)
3. cn=PUBLIC (această intrare este de obicei un container special sau o intrare pentru acces public/anonim, nu reprezintă un utilizator individual în sensul tipic al cuvântului, precum un cont de persoană sau de administrator)

Astfel, dintre intrările vizibile direct sub cn=Users, cele care reprezintă **utilizatori individuali sau conturi administrative distincte** sunt:

- cn=erednic
- cn=orcladmin

Prin urmare, sunt **2** utilizatori în acest container, conform interpretării standard a unui "user" în contextul unui director.

Întrebarea 111

5. Fișierul add_user.ldif

(Imaginea cu conținutul fișierului add_user.ldif)

Pentru ce tip de operație LDAP este folosit?

- a. Ldapadd
- b. Ldapadddn
- c. Ldapaddn
- d. LdapDnp

Răspuns: a. Ldapadd

Explicație:

Fișierul .ldif (LDAP Data Interchange Format) prezentat conține o intrare specifică:

- dn: cn=erednic,cn=users,dc=erednic,dc=com
- changetype: add

- Urmează o serie de attribute (cn, sn, userpassword, mail) și clase de obiect (objectclass).

Linia cheie aici este **changetype: add**. Aceasta indică explicit că operația care urmează să fie efectuată pe serverul LDAP este o operație de **adaugare** a unei noi intrări în director.

Comanda standard de linie de comandă în LDAP, utilizată pentru a adăuga intrări noi într-un director dintr-un fișier LDIF (care conține changetype: add), este **ldapadd**.

Celelalte opțiuni (Ldapadddn, Ldapaddn, LdapDnp) nu sunt comenzi standard recunoscute în ecosistemul LDAP. Ldapmodrdn este folosită pentru a modifica numele distinct relativ (RDN) al unei intrări, iar ldapmodify poate fi folosită pentru a adăuga, modifica sau șterge attribute dintr-o intrare existentă sau pentru a modifica DN-uri, dar pentru adăugarea unei intrări noi, ldapadd este comanda specifică și cea mai potrivită.

Întrebarea 112

6. Avem fișierul add_user.ldif

(Imaginea cu conținutul fișierului add_user.ldif)

Ce objectclass ar mai trebui adăugat?

- Person
- inetOrgPerson, organisationalPerson, orcluser
- orcluser, orcluserv2
- Person, inetOrgPerson, organisationalPerson, orcluser, orcluserv2

Răspuns: d. Person, inetOrgPerson, organisationalPerson, orcluser, orcluserv2

Explicație:

Fișierul LDIF dat descrie o intrare de utilizator cu attribute precum cn (common name), sn (surname), userpassword și mail. Pentru ca o astfel de intrare să fie validă și completă într-un director LDAP, în special într-un Oracle Internet Directory (OID), este nevoie de mai multe clase de obiecte (objectclass) pe lângă top, care este întotdeauna obligatorie.

Să analizăm clasele de obiect necesare și contextul lor:

1. **top**: Este clasa de obiect rădăcină, obligatorie pentru aproape toate intrările LDAP. Este deja inclusă în fișierul LDIF.
2. **person**: Această clasă de obiect definește attributele de bază pentru o persoană, cum ar fi cn și sn. Având în vedere că cn: test1 și sn: test1 sunt prezente, person este necesară.

3. **organizationalPerson:** Această clasă de obiect structurală moștenește de la `person` și adaugă atribute relevante pentru o persoană în cadrul unei organizații (ex: title, description). Este adesea o clasă intermediară în ierarhia claselor pentru utilizatori.
4. **inetOrgPerson:** Această clasă de obiect structurală moștenește de la `organizationalPerson` și este cea mai comună clasă de obiect pentru reprezentarea utilizatorilor "internet-enabled". Aceasta definește atribute precum mail și userpassword, care sunt prezente în fișierul LDIF.
5. **orcluser și orcluserV2:** Acestea sunt clase de obiecte specifice Oracle Internet Directory (OID) care extind schema standard LDAP pentru a include atribute și funcționalități specifice OID necesare pentru gestionarea utilizatorilor în mediul Oracle.

Prin urmare, pentru a defini complet o intrare de utilizator cu atributele date în OID, este necesar să se includă toate aceste clase de obiecte în declarația `objectclass`. Opțiunea **d** le listează pe toate, reprezentând setul complet și corect de `objectclass` care ar trebui adăugate pe lângă top pentru un utilizator tipic în OID.

Întrebarea 113

Care comandă de mai jos returnează primul user din containerul Users din realm-ul: dc=erednic,dc=com?

- a. `ldapsearch -h host -p port -D "cn=orcladmin" -w password -z 1-b "cn=users"`
- b. `ldapsearch -h host -p port -D "cn=orcladmin" -w password -b "cn=users" -s one`
- c. `ldapsearch -h host -p port -D "cn=orcladmin" -w password -z 1-b "cn=users,dc=erednic,dc=com" -s one "objectclass=*" dn`
- d. `ldapsearch -h host -p port -D "cn=orcladmin" -w password -t 1-b "cn=users"`

Răspuns: c. `ldapsearch -h host -p port -D "cn=orcladmin" -w password -z 1-b "cn=users,dc=erednic,dc=com" -s one "objectclass=*" dn`

Explicație:

Pentru a găsi primul utilizator (sau, mai precis, prima intrare) dintr-un container LDAP specific, folosim comanda `ldapsearch` cu anumite opțiuni esențiale:

- **-h host -p port:** Specifică serverul și portul LDAP la care să te conectezi.
- **-D "cn=orcladmin" -w password:** Specifică Distinguished Name (DN) și parola pentru autentificare. Aici, ne autentificăm ca `orcladmin` pentru a avea permisiuni de căutare.
- **-b "base_dn":** Definește punctul de plecare al căutării în arborele de directoare.

- Pentru a căuta în containerul Users din realm-ul dc=erednic,dc=com, baza corectă de căutare este **cn=users,dc=erednic,dc=com**. Opțiunile a, b și d folosesc doar -b "cn=users", ceea ce ar fi o bază incorectă sau incompletă pentru realm-ul specificat.
- **-s scope:** Definește domeniul de căutare.
 - **one:** Caută doar intrările direct sub base_dn (un singur nivel în jos). Acest lucru este esențial pentru a găsi utilizatori care sunt copii direcți ai containerului cn=users.
 - sub (implicit dacă nu este specificat, ca în opțiunea a) ar căuta în întregul subarboare, ceea ce ar putea returna și intrări din sub-container.
 - base ar căuta doar intrarea cn=users însăși.
- **-z size_limit:** Limitează numărul de rezultate returnate.
 - **-z 1:** Asigură că se returnează doar prima intrare găsită, îndeplinind cerința de a obține "primul user".
- **"objectclass=*":** Acesta este filtrul de căutare. objectclass=* înseamnă că se caută orice intrare, indiferent de clasa ei de obiect. Acest lucru este suficient pentru a găsi orice intrare directă din container.
- **dn:** Specifică attributele pe care dorești să le vezi în rezultate. Aici, cerem doar Distinguished Name (DN) al intrării găsite.

Analizând opțiunile:

- a. și b. au o bază de căutare (-b "cn=users") incorectă pentru realm-ul specificat.
- d. folosește o opțiune incorectă -t 1 în loc de -z 1 pentru limitarea rezultatelor și are, de asemenea, o bază de căutare incorectă.
- c. include toate elementele corecte: baza de căutare completă și corectă (cn=users,dc=erednic,dc=com), scopul de căutare de un nivel (-s one), limitarea la un singur rezultat (-z 1), un filtru valid ("objectclass=*") și specificarea de a returna DN-ul.

Întrebarea 114

Ldapcreate

- a. E folosit pentru crearea unui nou entry în LDAP
- b. E utilizat pentru crearea a uneia sau mai multe entry-uri în LDAP

c. Ajută la ștergerea unui entry sau mai multor

d. Nu este comandă LDAP

Răspuns: d. Nu este comandă LDAP

Explicație:

În setul de unelte standard pentru gestionarea unui director LDAP, nu există o comandă de linie numită "Ldapcreate". Operația de creare a unei noi intrări (entry) într-un director LDAP se realizează folosind comanda standard **ldapadd**. Această comandă este folosită pentru a adăuga una sau mai multe intrări noi, de obicei citind datele dintr-un fișier LDIF.

Prin urmare, deoarece "Ldapcreate" nu este o comandă LDAP recunoscută, afirmația corectă este că nu este o comandă LDAP.